

UVA CS 4774: Machine Learning

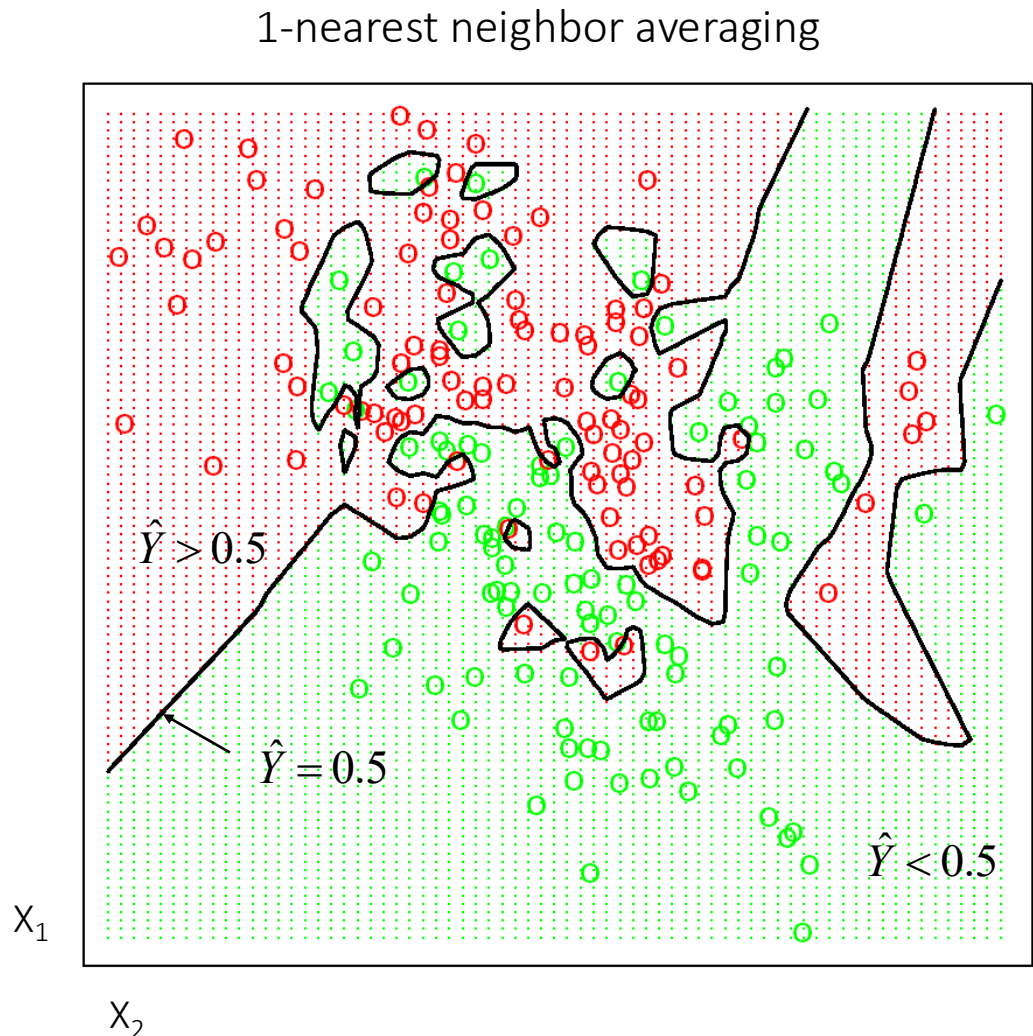
Lecture 9: Bias-Variance Tradeoff

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Lesson Learned : Training Error from KNN

- When $k = 1$,
- No misclassifications (on training): **Overfit**
- Minimizing training error is not always good (e.g., 1-NN)



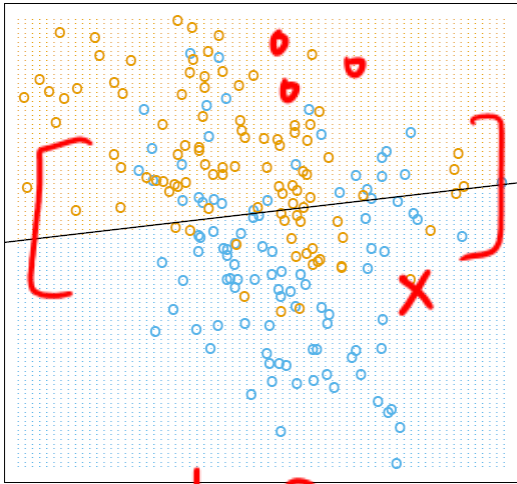
Complexity / Goodness of Fit / Generalization:

Decision boundaries in global vs. local models

$y \in \{\text{orange, blue}\}$

K=15

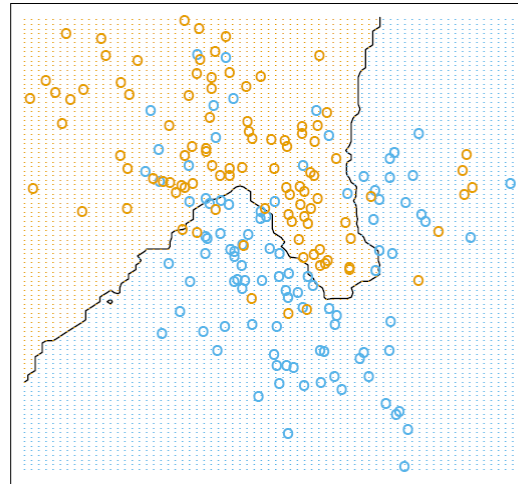
K=1



underfit

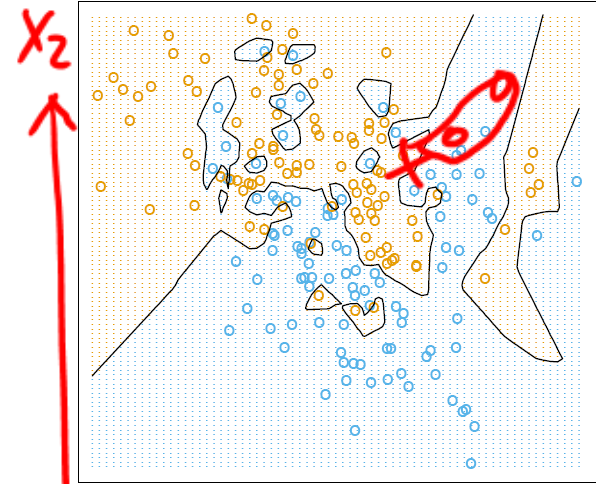
Linear classification

- global
- stable
- can be inaccurate



15-nearest neighbor

- K acts as a smoother



1-nearest neighbor

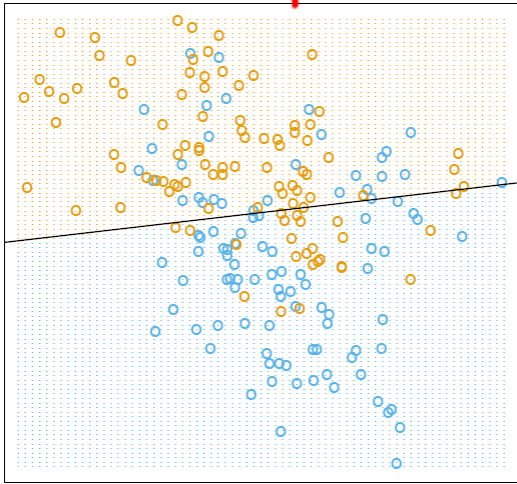
- local
- accurate
- unstable

overfitting

What ultimately matters: GENERALIZATION

Decision boundaries in Linear vs. kNN models (Later)

underfit

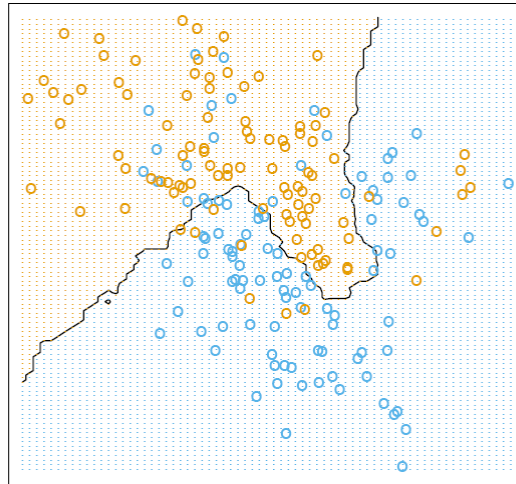


Low Variance /
High Bias

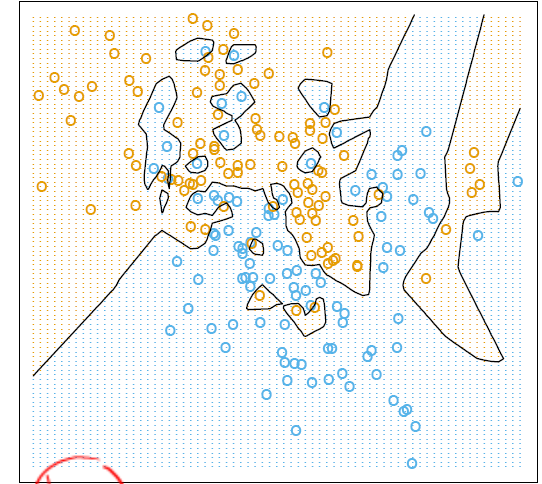
linear classifier

- global
- stable
- can be inaccurate

15-nearest neighbor



overfit



$k=1$

1-nearest neighbor

Low Bias
/ High Variance

KNN

- local
- accurate
- unstable

What ultimately matters: GENERALIZATION



Polynomial Regression

$d \nearrow$ Complexity $\nearrow$
 $d \searrow$ Complexity $\searrow$

Ridge Regression

$\lambda \nearrow$ Complexity $\searrow$
 $\lambda \searrow$ Complexity $\nearrow$

KNN

$k \nearrow$ Complexity $\searrow$
 $k \searrow$ Complexity $\nearrow$

poly Reg

$d \nearrow$

Ridge

$\lambda \searrow$

kNN

$k \searrow$

model
complex
small

model complexity
large

Less complex

More complex



① Polynomial Regression

d

$d \downarrow$

$d \uparrow$

② Ridge Reg

$\lambda > 0$

$\lambda \uparrow$

$\lambda \downarrow$

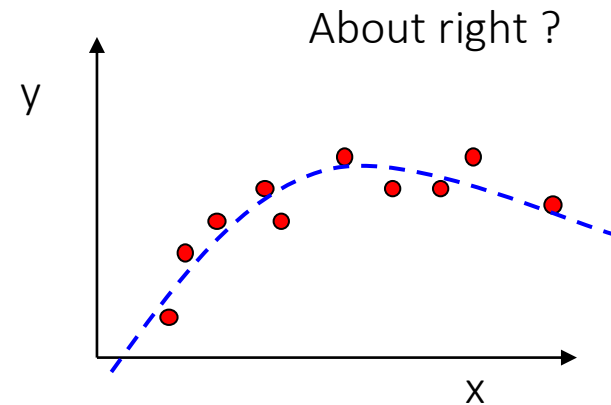
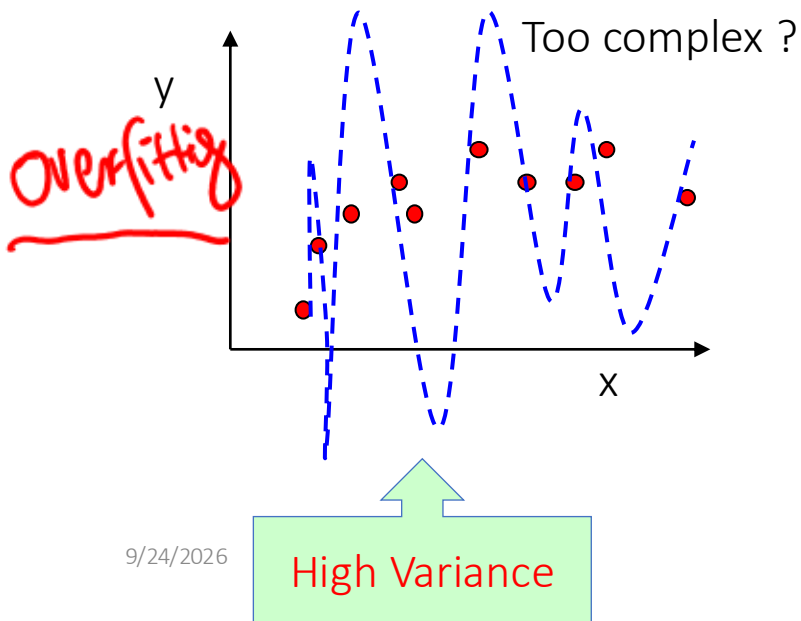
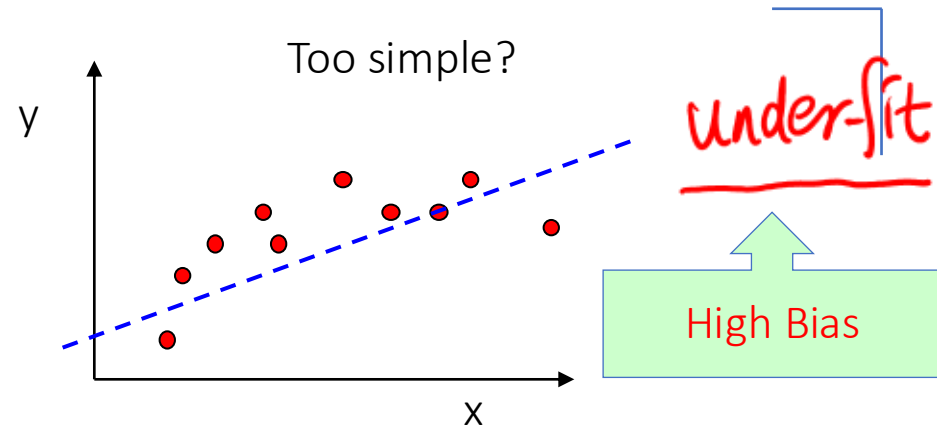
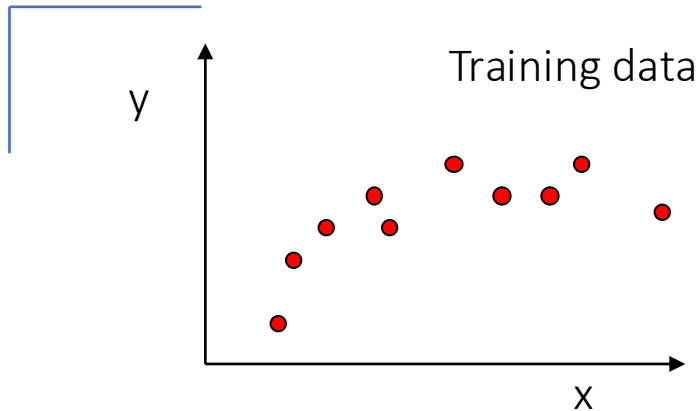
③ KNN Reg

k

$k \uparrow$

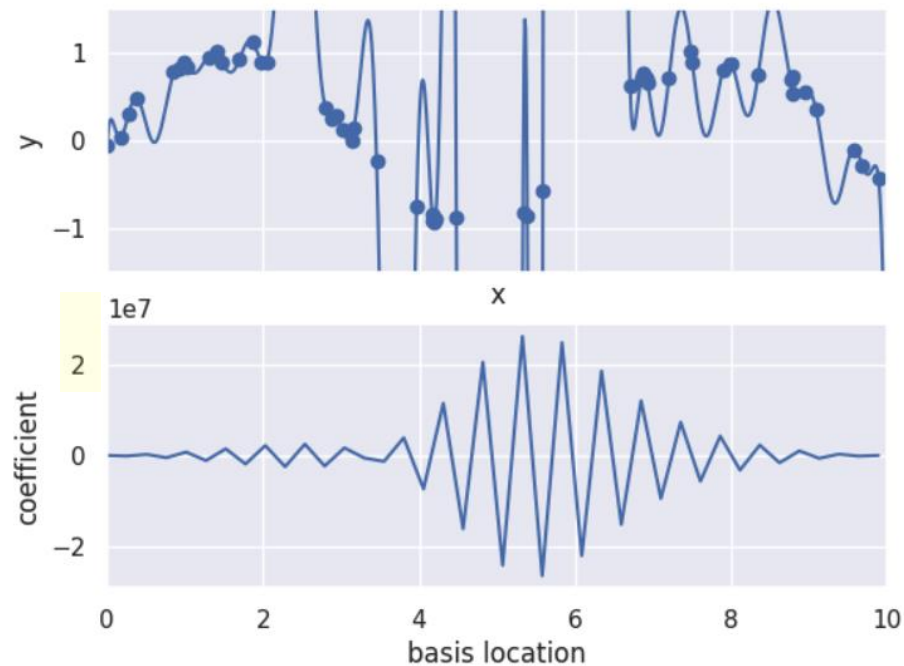
$k \downarrow$

Complexity / Goodness of Fit / Generalization



What ultimately matters: GENERALIZATION

```
model = make_pipeline(GaussianFeatures(40), LinearRegression())
basis_plot(model)
```



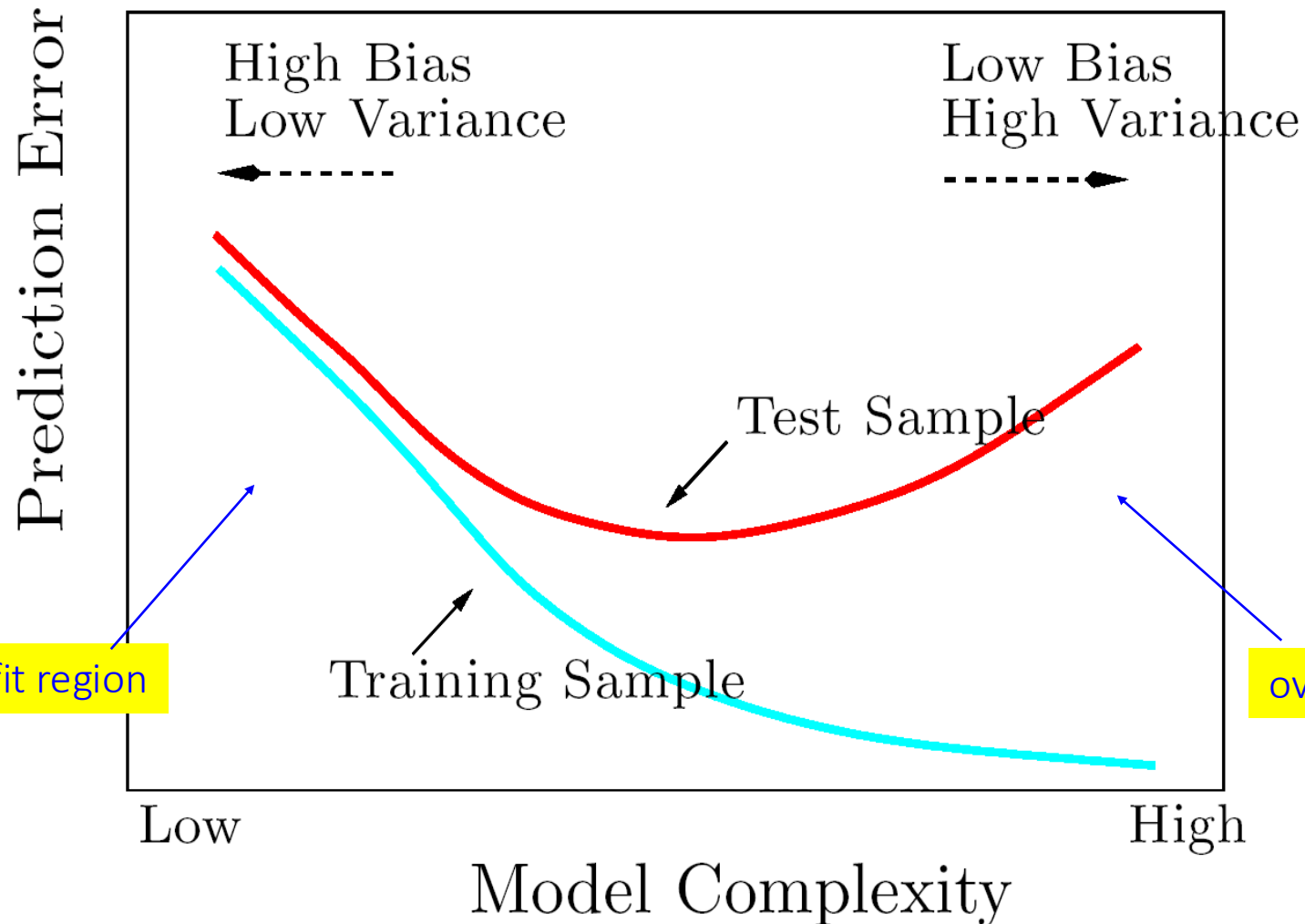
Regularization of parameters

➔ make the learned model weights more REGULAR

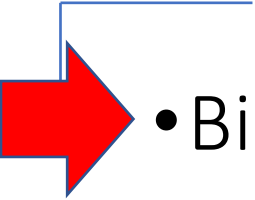
```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(40), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
```



Bias-Variance Tradeoff / Model Selection



Roadmap



- Bias-variance decomposition
- Bias-Variance Tradeoff / Model Selection
- Remedy when Overfit / Underfit

Review: Mean and Variance of Random Variable (RV)

X : random variables written in capital letter

$\Rightarrow \vec{X} = [X_1, X_2, \dots, X_p]$ random vector

$\Rightarrow Y$ random variable

$\Rightarrow f(\vec{X})$ random variable
e.g. $\theta^T \vec{X}$

$\Rightarrow \vec{\theta}$ random variable

Review: Mean and Variance of Random Variable (RV)

- Mean (Expectation):

- Discrete RVs:

$$E(X) = \sum v_i * P(X = v_i) = (-1) \times 0.1 + 1 \times 0.9$$

$$E(t) = -0.1 + 0.9 = 0.8$$

- Continuous RVs:

$$E(X) = \int_{-\infty}^{+\infty} x * p(x) dx$$

$$X \sim N(2, \sigma)$$
$$E(X) = 2$$

Bernoulli: $t \in \{-1, 1\}$

$$P(t = -1) = 0.1$$

$$P(t = 1) = 0.9$$

Review: Mean and Variance of Function of Random Variable (RV)

Bernoulli: $t \in \{-1, 1\}$
 $\begin{cases} p(t=-1)=0.1 \\ p(t=1)=0.9 \end{cases}$

$$\Rightarrow g(t) = 2t$$

- Mean (Expectation):

- Discrete RVs:

$$E(g(t)) = 2 \times (-1) \times 0.1 + 2 \times (1) \times 0.9 = \underline{1.6}$$

$$E[aX] = aE[X]$$

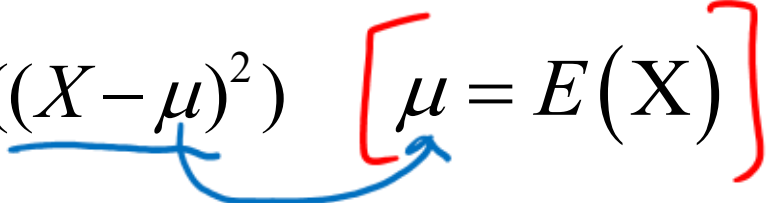
$$E(g(X)) = \sum_{v_i} \underline{g(v_i)} P(X = v_i)$$

- Continuous RVs:

$$E(g(X)) = \int_{-\infty}^{+\infty} g(x) * p(x) dx$$

Review: Mean and Variance of RV

• Variance: $Var(X) = E((X - \mu)^2)$ [$\mu = E(X)$]



• Discrete RVs: $V(X) = \sum_{v_i} (v_i - \mu)^2 P(X = v_i)$

• Continuous RVs:

$$V(X) = \int_{-\infty}^{+\infty} (x - \mu)^2 p(x) dx$$

Statistical Decision Theory

- Random input vector: X
- Random output variable: Y
- Joint distribution: $\Pr(X, Y)$
- Loss function $L(Y, f(X))$
- Expected prediction error (EPE):

$$\text{EPE}(f) = E(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

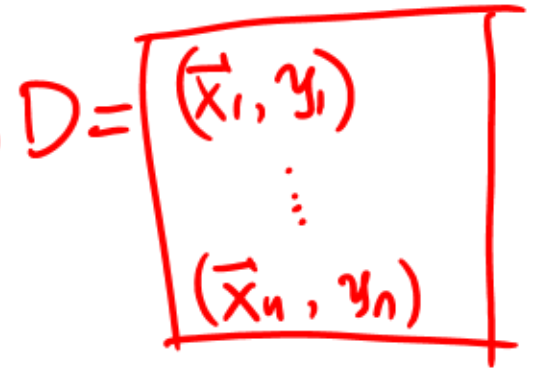
$$\text{e.g.} = \int (y - f(x))^2 \Pr(dx, dy)$$

e.g. Squared error loss (also called L2 loss)

Consider
population
distribution

Statistical Decision Theory (Extra)

- Random input vector: X
- Random output variable: Y
- Joint distribution: $\Pr(X, Y)$
- Loss function $L(Y, f(X))$


$$\Rightarrow D = \begin{pmatrix} (\vec{x}_1, y_1) \\ \vdots \\ (\vec{x}_n, y_n) \end{pmatrix}$$

- Expected prediction error (EPE):

$$\text{EPE}(f) = E(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

$$\text{e.g.} = \int (y - f(x))^2 \Pr(dx, dy)$$

e.g. Squared error loss (also called L2 loss)

Consider
population
distribution

Test Error to EPE: (Extra)

- (Almost) the same definition:
- Expected Prediction Error:
 - Expected Test Error:
 - Expected Risk of a hypothesis
- ➔ Empirical Risk Minimization

• Expected prediction error (EPE): $\sum_{i=1}^n L(y_i, f(x_i))$

$$\text{EPE}(f) = E(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

e.g. = $\int (y - f(x))^2 \Pr(dx, dy)$

e.g. Squared error loss (also called L2 loss)

One way to define generalization: by considering the joint population distribution

Decomposition of EPE

- When additive error model: $Y = \underline{f}(X) + \epsilon, \epsilon \sim (0, \sigma^2)$
- Notations
 - Output random variable: Y
 - True function: $f \rightarrow \text{true}$
 - Prediction estimator: $\hat{f} \rightarrow D \rightarrow \hat{f}$

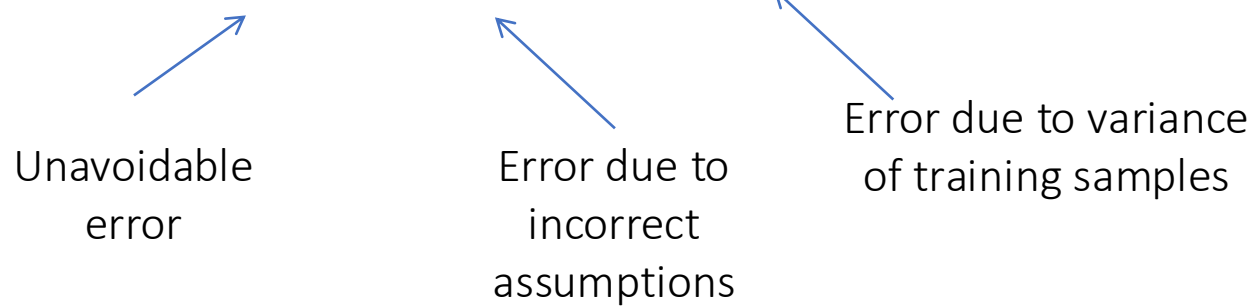
$$\begin{aligned} EPE(x) &= E[(Y - \hat{f})^2 | X = x] \\ &= E[((Y - f) + (f - \hat{f}))^2 | X = x] \\ &= \underbrace{E[(Y - f)^2 | X = x]}_{\epsilon} + \underbrace{E[(f - \hat{f})^2 | X = x]}_{\text{Bayes Error}} \\ &= \sigma^2 + \underbrace{Var(\hat{f}) + Bias^2(\hat{f})}_{\text{Bayes Error}} \end{aligned}$$

Irreducible / Bayes error

Bias-Variance Trade-off for EPE:

$$\text{EPE}(x) = \text{noise}^2 + \text{bias}^2 + \text{variance}$$

Unavoidable
error



The diagram illustrates the Bias-Variance Trade-off for Expected Prediction Error (EPE). The equation $\text{EPE}(x) = \text{noise}^2 + \text{bias}^2 + \text{variance}$ is centered at the top. Three blue arrows point from descriptive text labels below to the terms in the equation: one from 'Unavoidable error' to 'noise²', one from 'Error due to incorrect assumptions' to 'bias²', and one from 'Error due to variance of training samples' to 'variance'.

Error due to
incorrect
assumptions

Error due to variance
of training samples

$$\left\{ \begin{array}{l} Y = f(x) + \underline{\varepsilon} \quad \varepsilon \sim N(0, \sigma^2) \\ \hat{f}_D(x) \\ E_D(\hat{f}(x)) = \int_D \hat{f}_D(x) \underline{p(D)} \end{array} \right. \Rightarrow \begin{array}{l} \underline{f^{\text{true}}} \\ \hat{f}^{\text{estimated}} \\ \underline{\bar{f}}^{\text{Expected Estimated}} \end{array}$$
$$\begin{aligned} \text{EPE} &\approx E_{(x, Y)} \left[(Y - \hat{f}(x))^2 \right] \\ &= E_{(x, Y)} \left[((Y - f) + (f - \hat{f}))^2 \right] \\ &= \underline{E[(Y - f)^2]} + E[(f - \hat{f})^2] \\ \text{Bayes error} &= \sigma^2 + \underline{E[(f - \hat{f})^2]} \end{aligned}$$

$$E[(f - \hat{f})^2]$$

$$= E\left[\underbrace{(f - \bar{f})}_{\text{Bias}^2} + \underbrace{(\bar{f} - \hat{f})}_{\text{Variance}}\right]^2$$

$$= E[(f - \bar{f})^2] + E[(\bar{f} - \hat{f})^2]$$

$$E[2(f - \bar{f})(\bar{f} - \hat{f})] = 0$$

$$E(\hat{f}) = \bar{f} \Rightarrow$$

~~$$2E[f\bar{f}] - 2E[f\hat{f}] - 2E[\bar{f}\bar{f}] + 2E[\bar{f}\hat{f}]$$~~

- More so than just these intuitive descriptions, the expected test error mathematically decomposes into a sum of three corresponding parts. Begin by writing the model

$$Y = f(X) + \varepsilon,$$

where ε has mean zero, variance σ^2 , and is independent of X . Note that the independence condition is the an actual (nontrivial) assumption. Recall that (x_i, y_i) , $i = 1, \dots, n$ are independent of each other and of (X, Y) , all with the same distribution. We'll look at the expected test error, conditional on $X = x$ for some arbitrary input x . It follows that

$$\mathbb{E}[(Y - \hat{f}(x))^2 | X = x] = \sigma^2 + \underbrace{\mathbb{E}[(f(x) - \hat{f}(x))^2]}_{\text{Risk}(\hat{f}(x))}. \quad \leftarrow$$

The first term σ^2 is the *irreducible error*, or sometimes referred to as the *Bayes error*, and the second term is called the risk, or mean squared error (MSE). The risk further decomposes into two parts, so that

$$\underbrace{\mathbb{E}[(Y - \hat{f}(x))^2 | X = x]}_{\text{EPE}} = \underbrace{\sigma^2}_{\text{BayesE}} + \underbrace{\mathbb{E}[(f(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Bias}^2(\hat{f}(x))} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Var}(\hat{f}(x))}, \quad (2)$$

true $f - \bar{f}(x)$
 $\mathbb{E}[(\hat{f} - \bar{f})^2]$

the latter terms being the squared *estimation bias* or simply *bias*, and the *estimation variance* or simply *variance*, respectively. The decomposition (2) is called the *bias-variance decomposition* or *bias-variance tradeoff*

$$E \left[\left(Y - \hat{f}(x) \right)^2 \right] = E \left[\left(f(X) + \epsilon - \hat{f}(x) \right)^2 \right]$$

$$= E \left[\left(f(X) - \hat{f}(x) \right)^2 \right] + 2E[\epsilon(f(x) - \hat{f}(x))] + E[\epsilon^2] = MSE(f, \hat{f}) + Var(\epsilon)$$

Assuming the Bayes error is independent of $\hat{f}(x)$,

$$E[\epsilon(f(x) - \hat{f}(x))] = E[\epsilon]E[f(x) - \hat{f}(x)] = 0$$

$$E[\epsilon^2] = \sigma^2 + E[\epsilon]^2 = \sigma^2$$

$$E \left[\left(f(X) - \hat{f}(x) \right)^2 \right] = E \left[\left((f(X) - E[\hat{f}(x)]) + (E[\hat{f}(x)] - \hat{f}(x)) \right)^2 \right]$$

$$\begin{aligned}
&= E \left[(f(X) - E[\hat{f}(x)])^2 + 2(f(X) - E[\hat{f}(x)])(E[\hat{f}(x)] - \hat{f}(x)) + (E[\hat{f}(x)] - \hat{f}(x))^2 \right] \\
&= E \left[(f(X) - E[\hat{f}(x)])^2 \right] + 2E \left[(f(X) - E[\hat{f}(x)])(E[\hat{f}(x)] - \hat{f}(x)) \right] + E \left[(E[\hat{f}(x)] - \hat{f}(x))^2 \right]
\end{aligned}$$

We can show:

$$2E \left[(f(X) - E[\hat{f}(x)])(E[\hat{f}(x)] - \hat{f}(x)) \right] = 2(f(X) - E[\hat{f}(x)])E[E[\hat{f}(x)] - \hat{f}(x)] = 0$$

Finally,

$$\begin{aligned}
E \left[(f(X) - \hat{f}(x))^2 \right] &= E \left[(f(X) - E[\hat{f}(x)])^2 \right] + E \left[(E[\hat{f}(x)] - \hat{f}(x))^2 \right] \\
&= \text{Bias}(f(x), \hat{f}(x))^2 + \text{Var}(\hat{f}(x))
\end{aligned}$$

Putting it all together:

$$E \left[(Y - \hat{f}(x))^2 \right] = \text{Bias}(f(x), \hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2$$

Another View: BIAS AND VARIANCE TRADE-OFF for parameter estimation (Extra)

$D_1 \rightarrow \hat{\theta}_1 / D_2 \rightarrow \hat{\theta}_2 / \dots / D_T \rightarrow \hat{\theta}_T$

$\bar{\theta} = \sum_{t=1}^T \hat{\theta}_t$

$$\begin{aligned} MSE(\hat{\theta}) &= E[(\hat{\theta} - \theta)^2] \\ &= E[(\hat{\theta} - \bar{\theta}) + (\bar{\theta} - \theta)]^2 \\ &= E[(\hat{\theta} - \bar{\theta})^2] + E[(\bar{\theta} - \theta)^2] + 2E[(\hat{\theta} - \bar{\theta})(\bar{\theta} - \theta)] \\ &= \underbrace{Var(\hat{\theta})}_{\text{Error due to variance of training samples}} + \underbrace{Bias^2(\hat{\theta})}_{\text{Error due to incorrect assumptions}} + 0 \end{aligned}$$

$E(\hat{\theta}) = \bar{\theta}$

Error due to variance of training samples

Error due to incorrect assumptions

$$MSE(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = Bias^2(\hat{\theta}) + Var(\hat{\theta})$$

$$MSE(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = Bias^2(\hat{\theta}) + Var(\hat{\theta})$$

BIAS AND VARIANCE TRADE-OFF for Parameter Estimation (Extra)

- θ : true value (normally unknown)
- $\hat{\theta}$: estimator
- $\bar{\theta} := E[\hat{\theta}]$ (mean, i.e. expectation of the estimator)
- Bias $E[(\bar{\theta} - \theta)^2]$
 - measures accuracy or quality of the estimator
 - low bias implies on average we will accurately estimate true parameter from training data
- Variance $E[(\hat{\theta} - \bar{\theta})^2]$
 - Measures precision or specificity of the estimator
 - Low variance implies the estimator does not change much as the training set varies

Model “bias” & Model “variance”

- Middle RED:

- TRUE function

θ
[middle red]

- Error due to bias:

- How far off in general from the middle red

$$E(\theta - \bar{\theta})$$

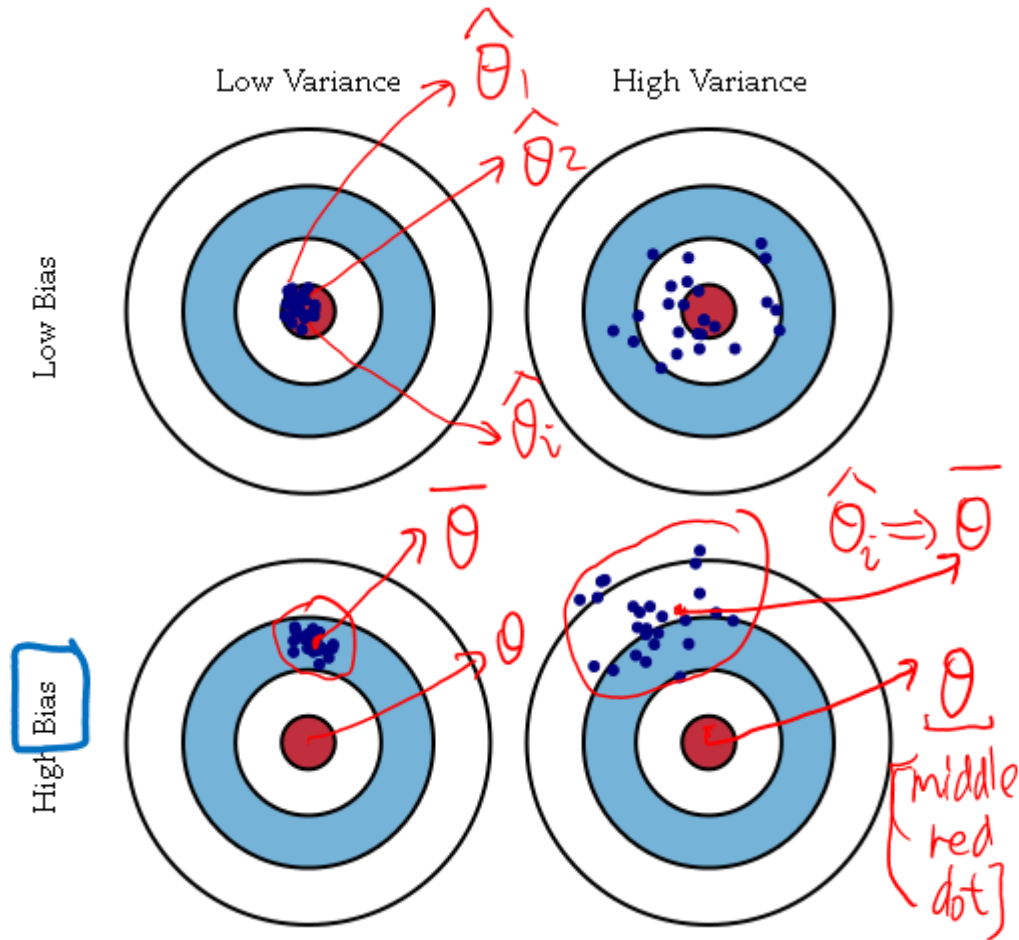
mean of $\hat{\theta}$

- Error due to variance:

- How wildly the blue points spread

$$E((\hat{\theta} - \bar{\theta})^2)$$

$\{\hat{\theta}_1, \hat{\theta}_2, \hat{\theta}_3, \dots\}$ Blue dots



Model “bias” & Model “variance”

- Middle RED:
 - TRUE function
- Error due to bias:
 - How far off in general from the middle red

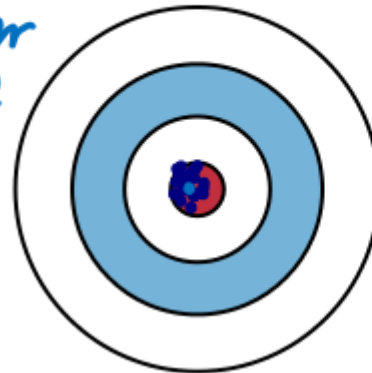
$$E[(\bar{\theta} - \theta)^2]$$

- Error due to variance:
 - How wildly the blue points spread

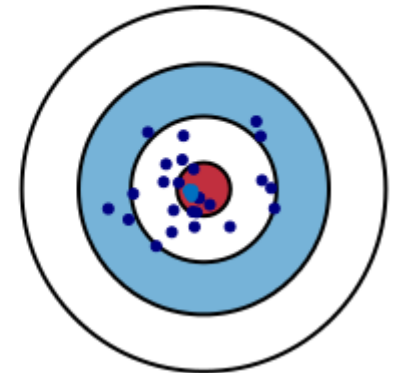
$$E[(\hat{\theta} - \bar{\theta})^2]$$

red
vs.
center
blue
Low Bias

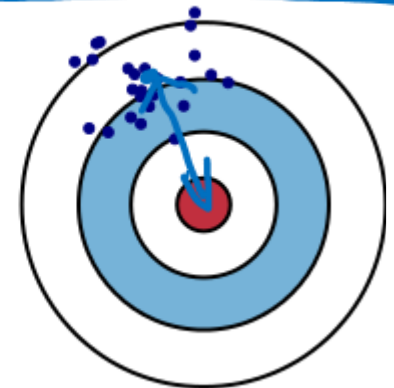
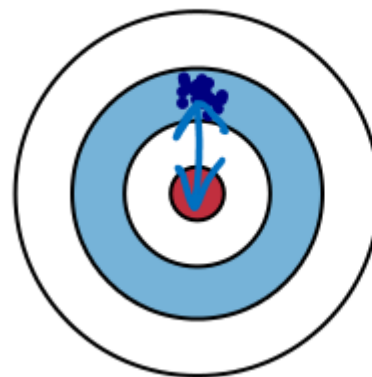
Low Variance



High Variance



High Bias



Model “bias” & Model “variance”

- Middle RED:
 - TRUE function

- Error due to bias:
 - How far off in general from the middle red

$$E[(\bar{\theta} - \theta)^2]$$

- Error due to variance:
 - How wildly the blue points spread

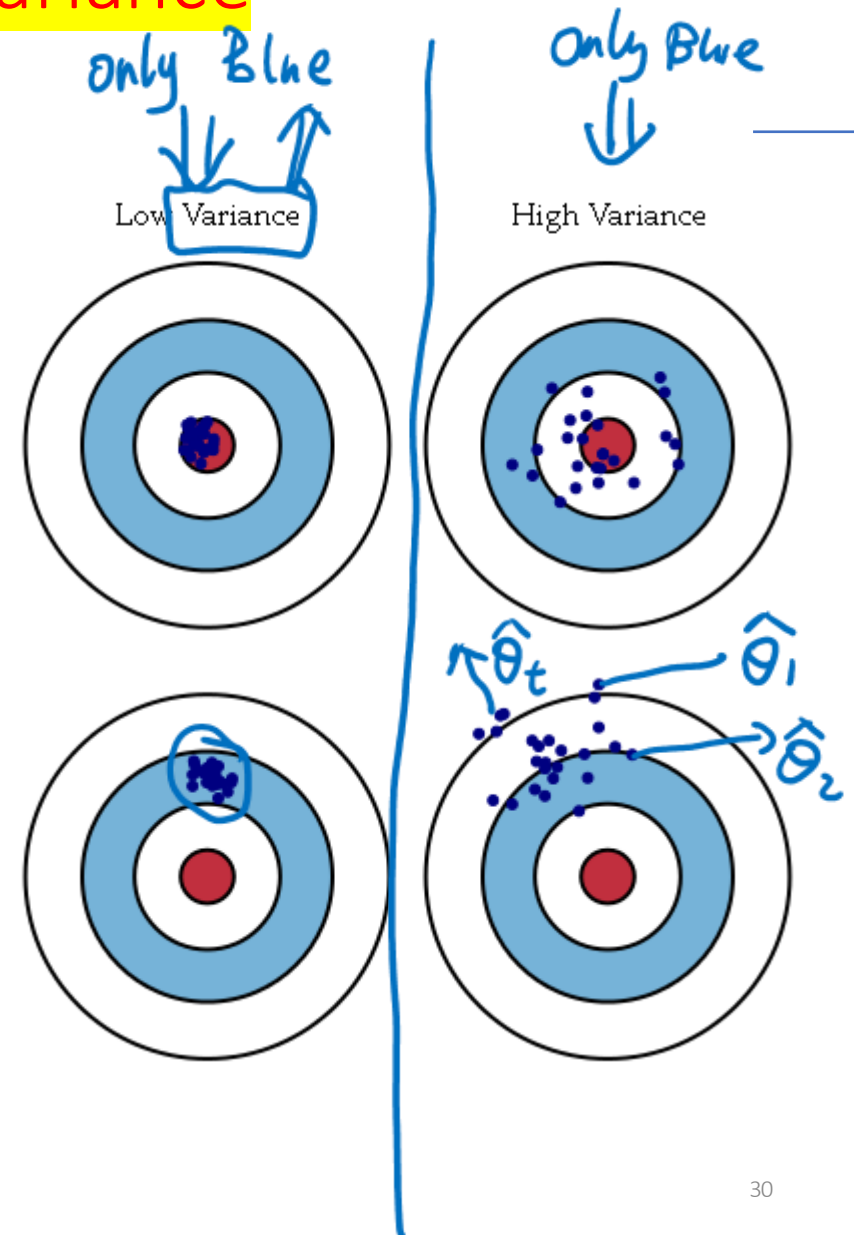
$$E[(\hat{\theta} - \bar{\theta})^2]$$

θ : red dot
 $\hat{\theta}_{it}$: blue dots

$\bar{\theta}$: mean point of all blue

Low Bias

High Bias



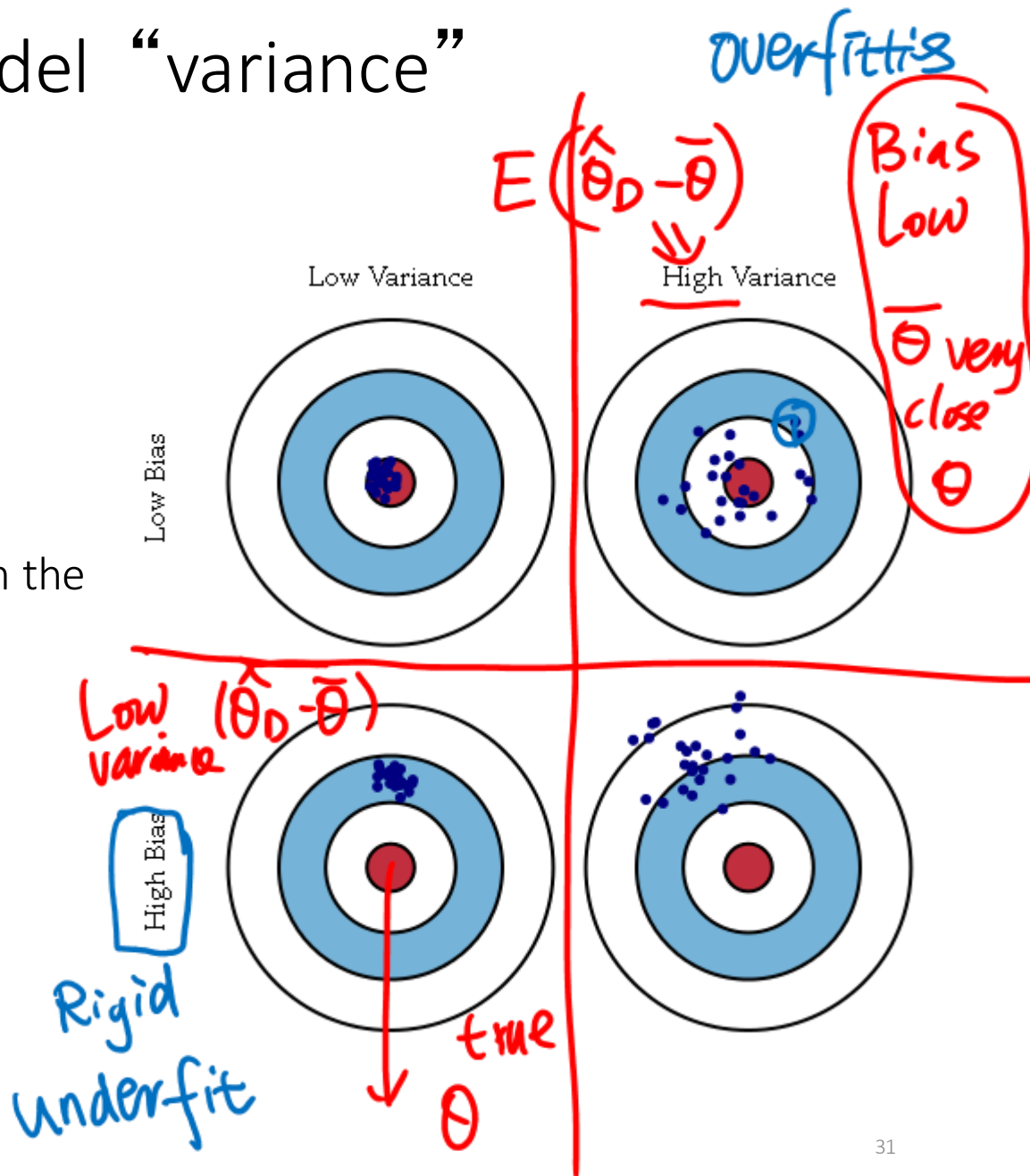
Model “bias” & Model “variance”

- Middle RED:
 - TRUE function
- Error due to bias:
 - How far off in general from the middle red

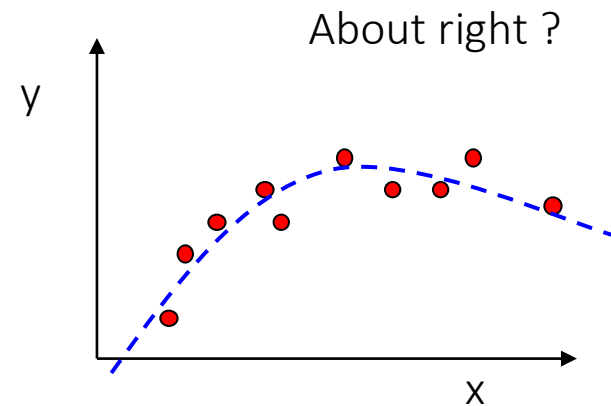
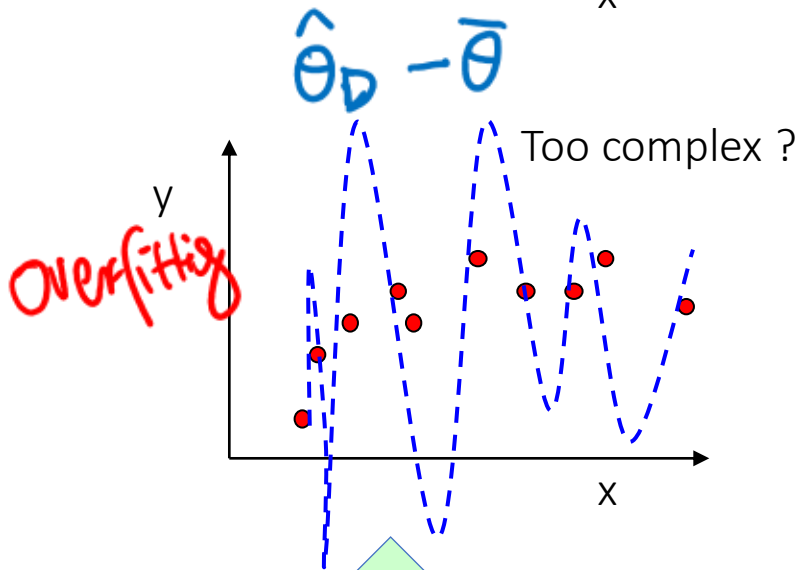
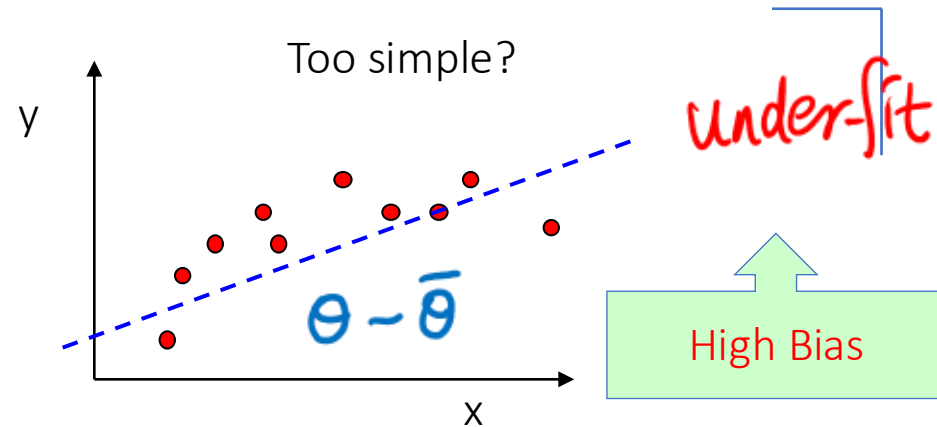
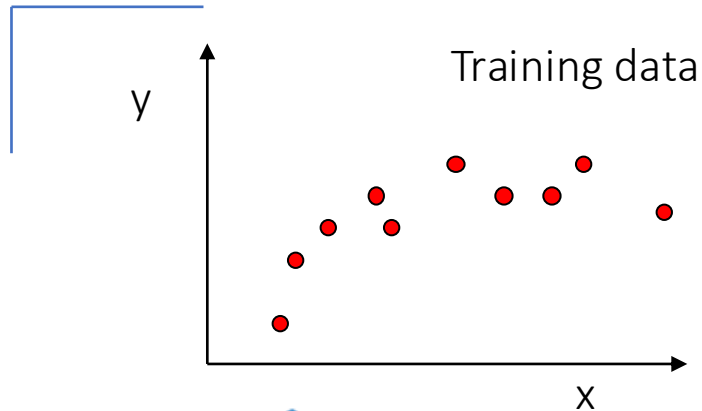
$$E[(\bar{\theta} - \theta)^2]$$

- Error due to variance:
 - How wildly the blue points spread

$$E[(\hat{\theta} - \bar{\theta})^2]$$



Complexity / Goodness of Fit / Generalization



What ultimately matters: GENERALIZATION

need to make assumptions that are able to generalize

- Underfitting: model is too “simple” to represent all the relevant characteristics
 - High bias and low variance
 - High training error and high test error
- Overfitting: model is too “complex” and fits irrelevant characteristics (noise) in the data
 - Low bias and high variance
 - Low training error and high test error

Thank You



UVA CS 4774: Machine Learning

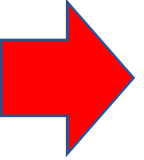
Lecture 9: Bias-Variance Tradeoff

Module II

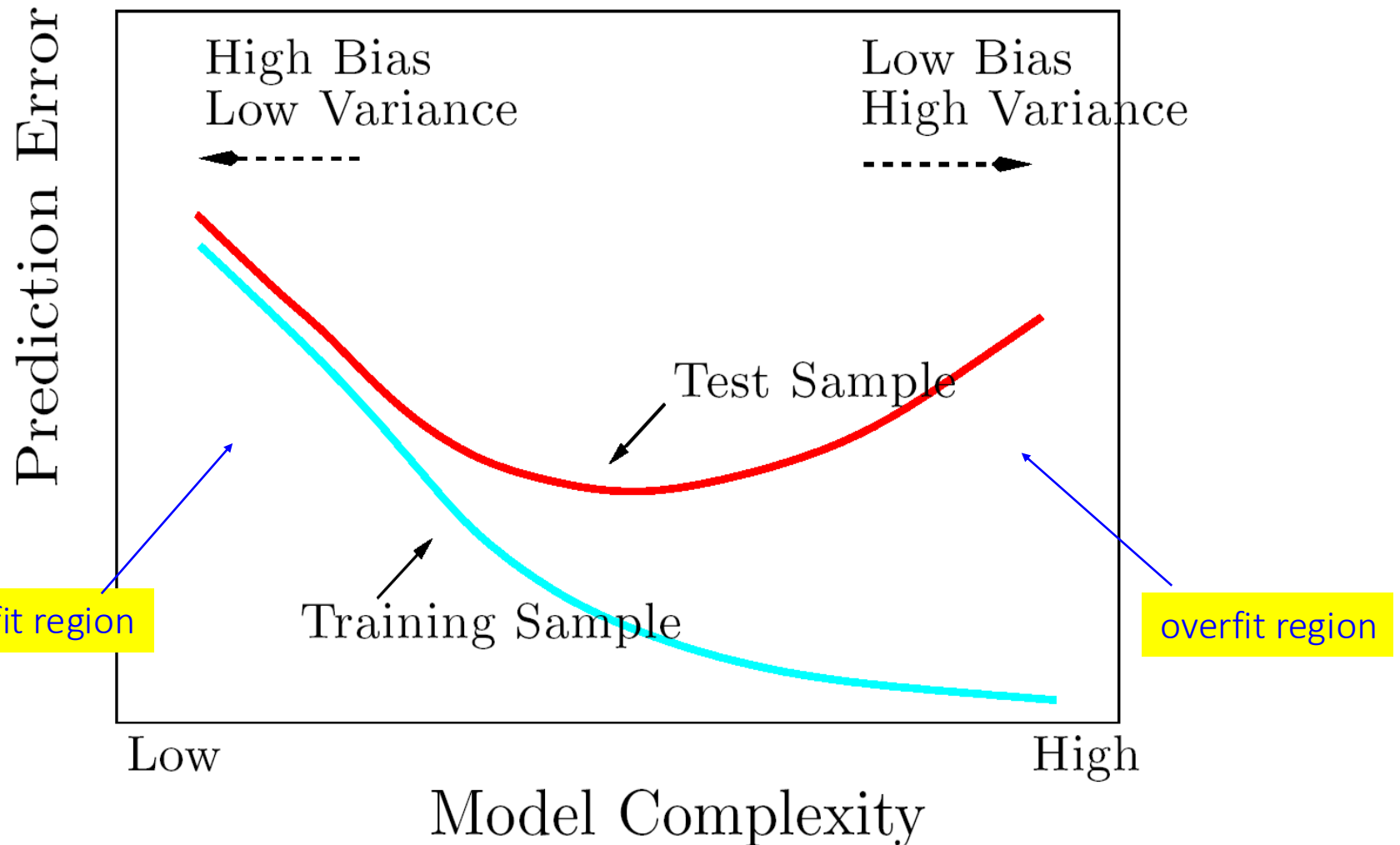
Dr. Yanjun Qi

University of Virginia
Department of Computer Science

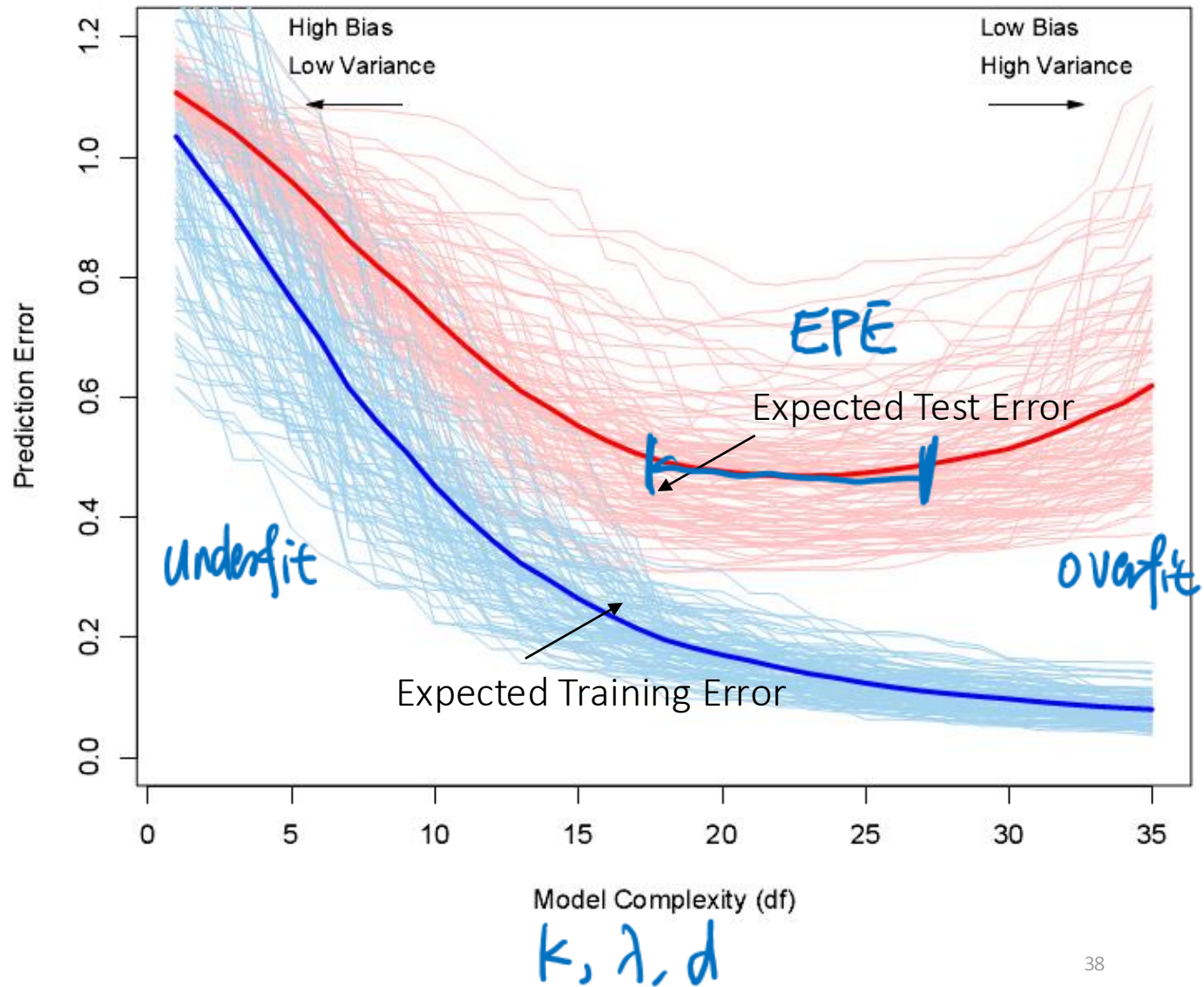
Roadmap

- Bias-variance decomposition
-  • Bias-Variance Tradeoff / Model Selection
- Remedy when Overfit / Underfit

Bias-Variance Tradeoff / Model Selection

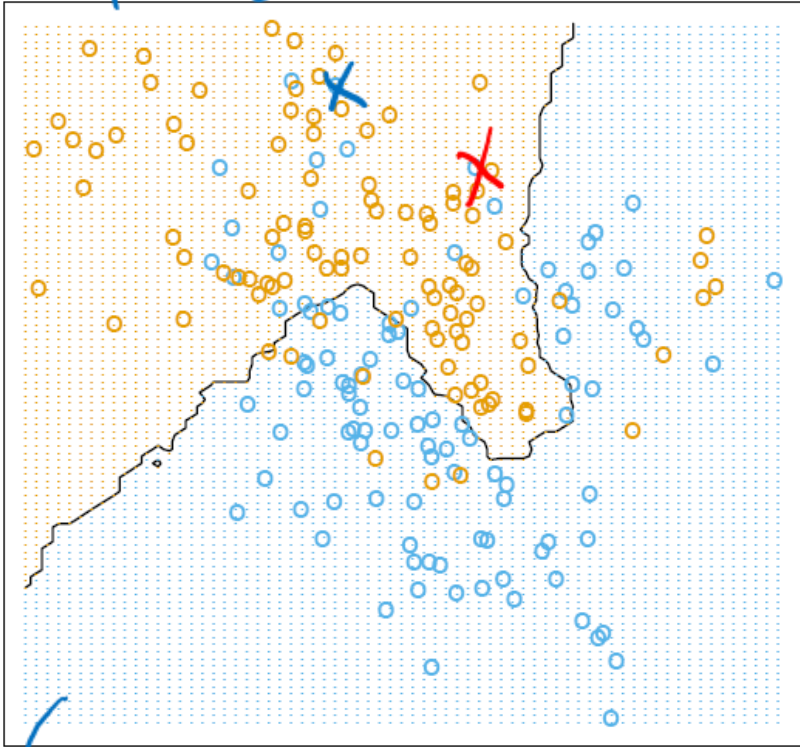


(1) Randomness of Training Sets $P_r(x, y)$

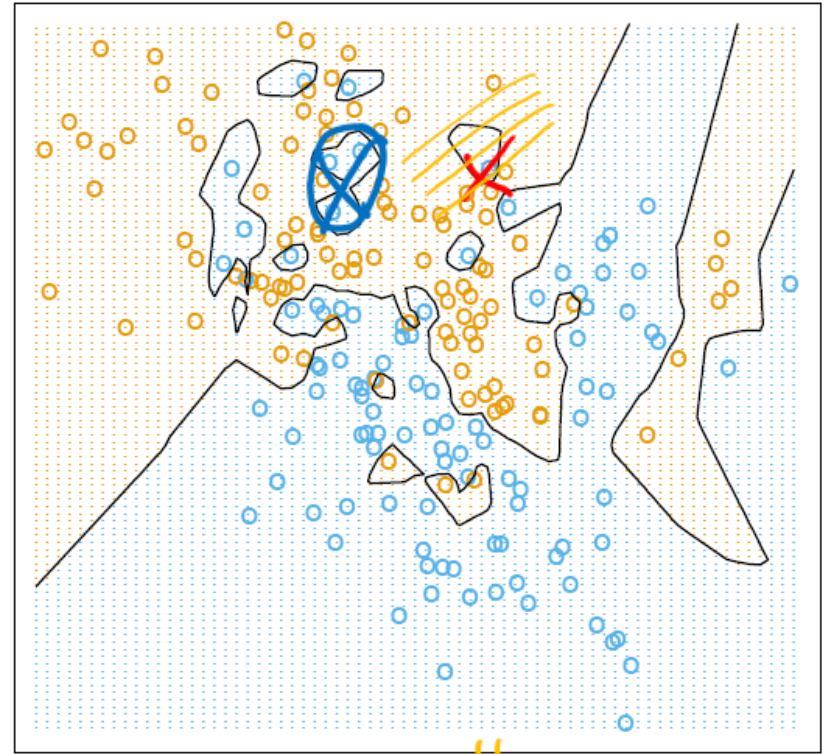


Randomness of Train Set => Variance of Models, e.g.,

$k=15$



$k=1$

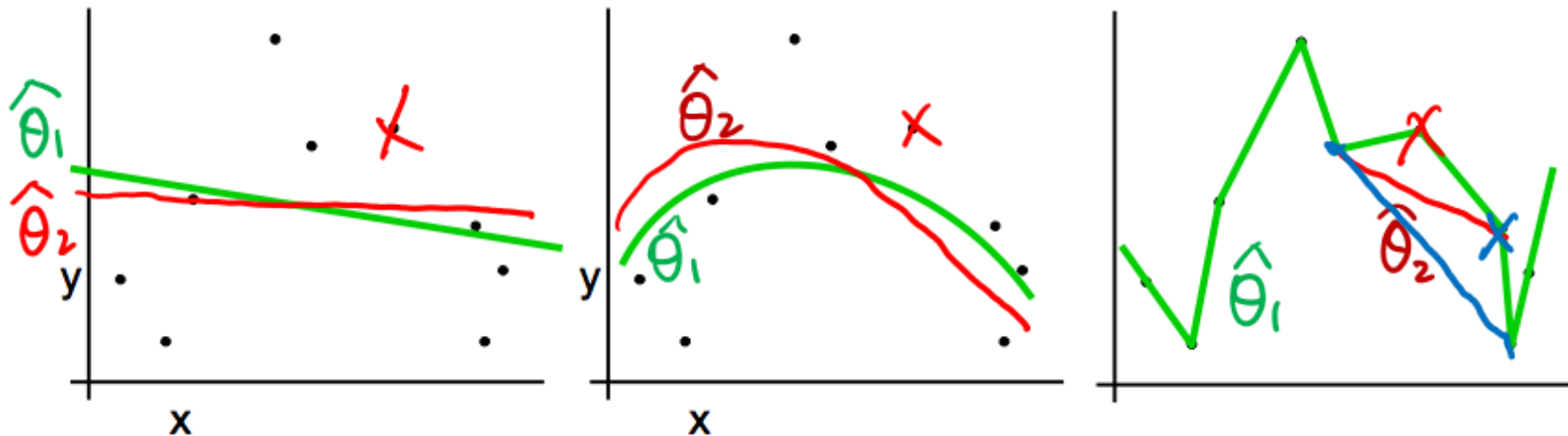


e.g. removing one train sample
[No change of decision boundary]

[decision boundary changed]

Randomness of Train Set

=> Variance of Models, e.g., for regression

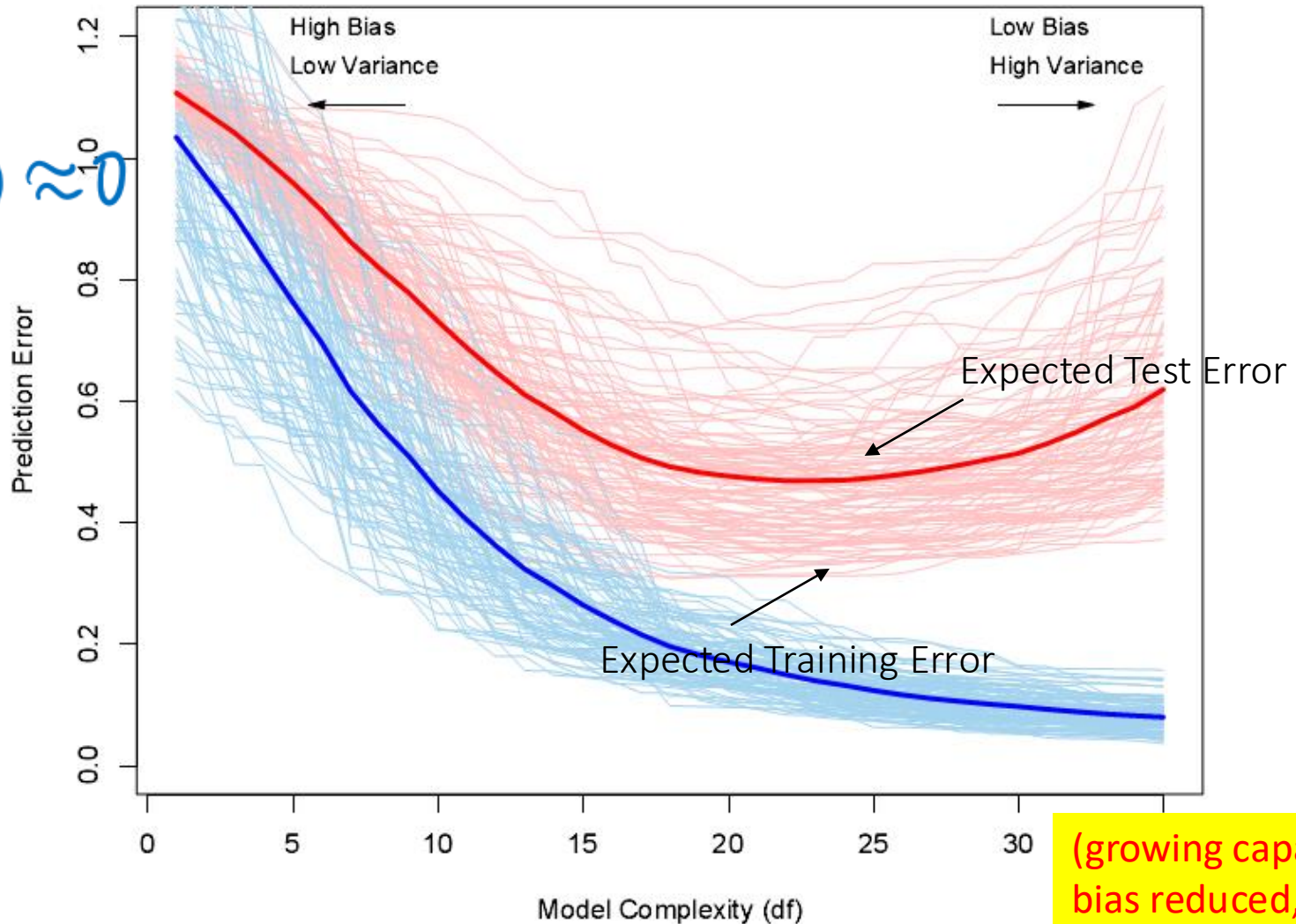


e.g. removing
one training sample

when model complexity $\uparrow$ $\Rightarrow$ model variance $\uparrow$

(2) Training error can always be reduced when increasing model complexity

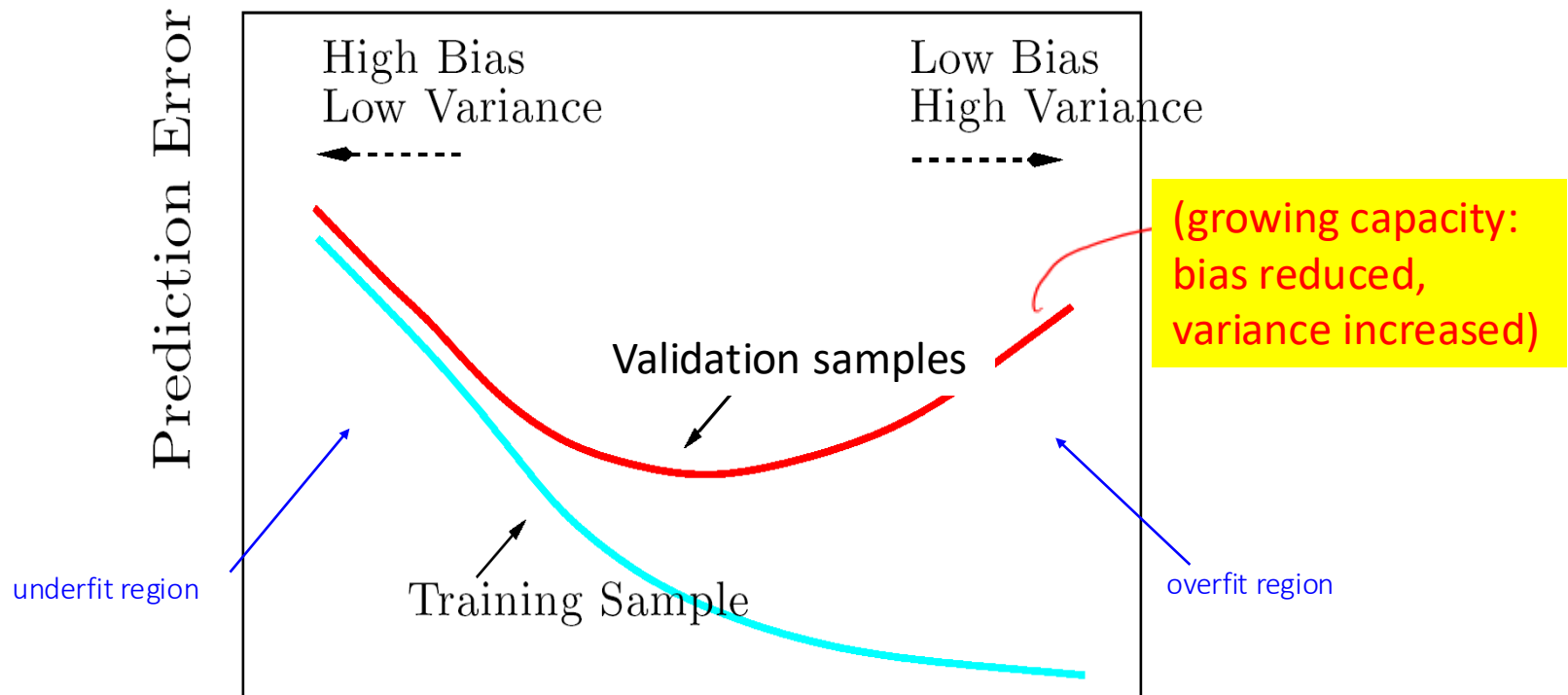
$k=1$
 $Tr(kNN) \approx 0$



(growing capacity:
bias reduced,
variance increased)

Bias-Variance Tradeoff / Model Selection

Rigid $\xrightarrow[k \downarrow \rightarrow, d \uparrow \rightarrow, \lambda \downarrow \rightarrow]{}$ Complex



KNN: large $k \leftarrow$ [Model Complexity] $\rightarrow$ small k
Regression: small $d \xrightarrow{\hspace{2cm}} \text{large } d$

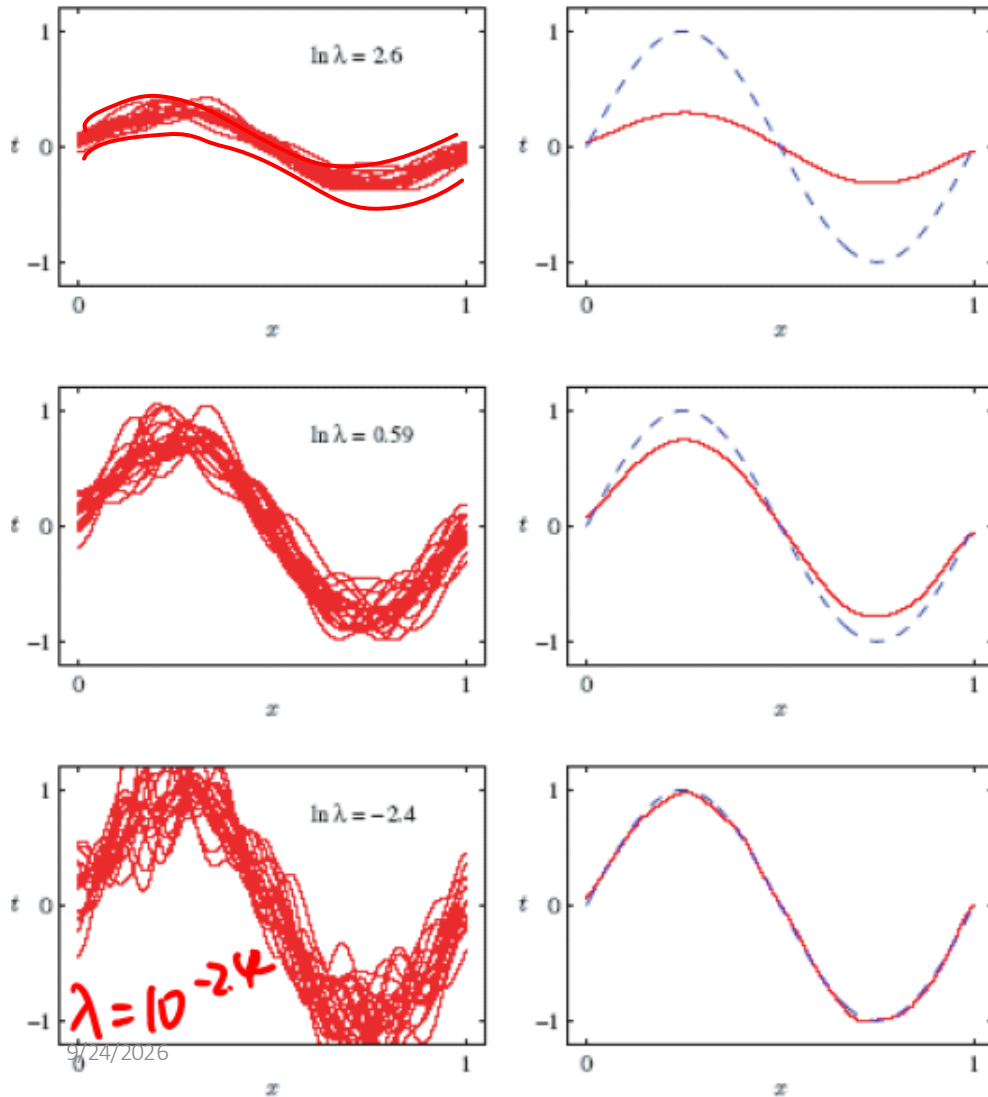
(4) Generalization Error as $\text{Bias}^2 + \text{variance}$ /
Model Selection? $\rightarrow$ Expected Testing Error

Generalization Error?
vs.
Testing Error?

e.g. Regularized LR as an example.

Here we simulate and know true θ

$$D_{ts} \rightarrow E[D_{ts}]$$



- lambda hyperparameter controls "regularization" terms in RLR, the smaller the lambda, is the more complex the model (why?)
 - Simple (highly regularized) models have low variance but high bias.
 - Complex models have low bias but high variance.
- You are inspecting an empirical average over 100 training set.

(growing capacity:
bias reduced,
variance increased)

$$MSE(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = Bias^2(\hat{\theta}) + Var(\hat{\theta})$$

BIAS AND VARIANCE TRADE-OFF for Parameter Estimation (Extra)

- θ : true value (normally unknown)
- $\hat{\theta}$: estimator
- $\bar{\theta} := E[\hat{\theta}]$ (mean, i.e. expectation of the estimator)

• Bias $E[(\bar{\theta} - \theta)^2]$

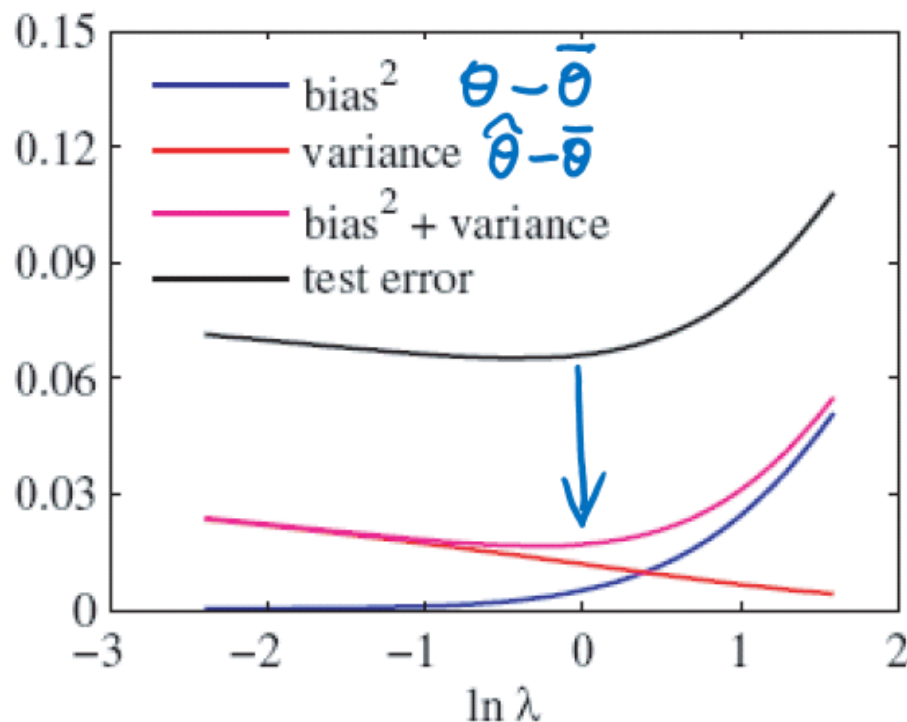
- measures **accuracy** or **quality** of the estimator
- low bias implies on average we will accurately estimate true **parameter** from training data

• Variance $E[(\hat{\theta} - \bar{\theta})^2]$

- Measures **precision** or **specificity** of the estimator
- Low variance implies the estimator does not **change** much as **the training set varies**

(4) Generalization Error as Bias²+variance / Model Selection? → Expected Testing Error

- bias decrease with model capacity,
- Variance increase with model capacity
- Sum of Bias²+Variance has a valley shape



- Bias²+variance matches (shape of) test error quite well.
- However, bias and variance cannot be computed since it relies on knowing the true distribution of x and y

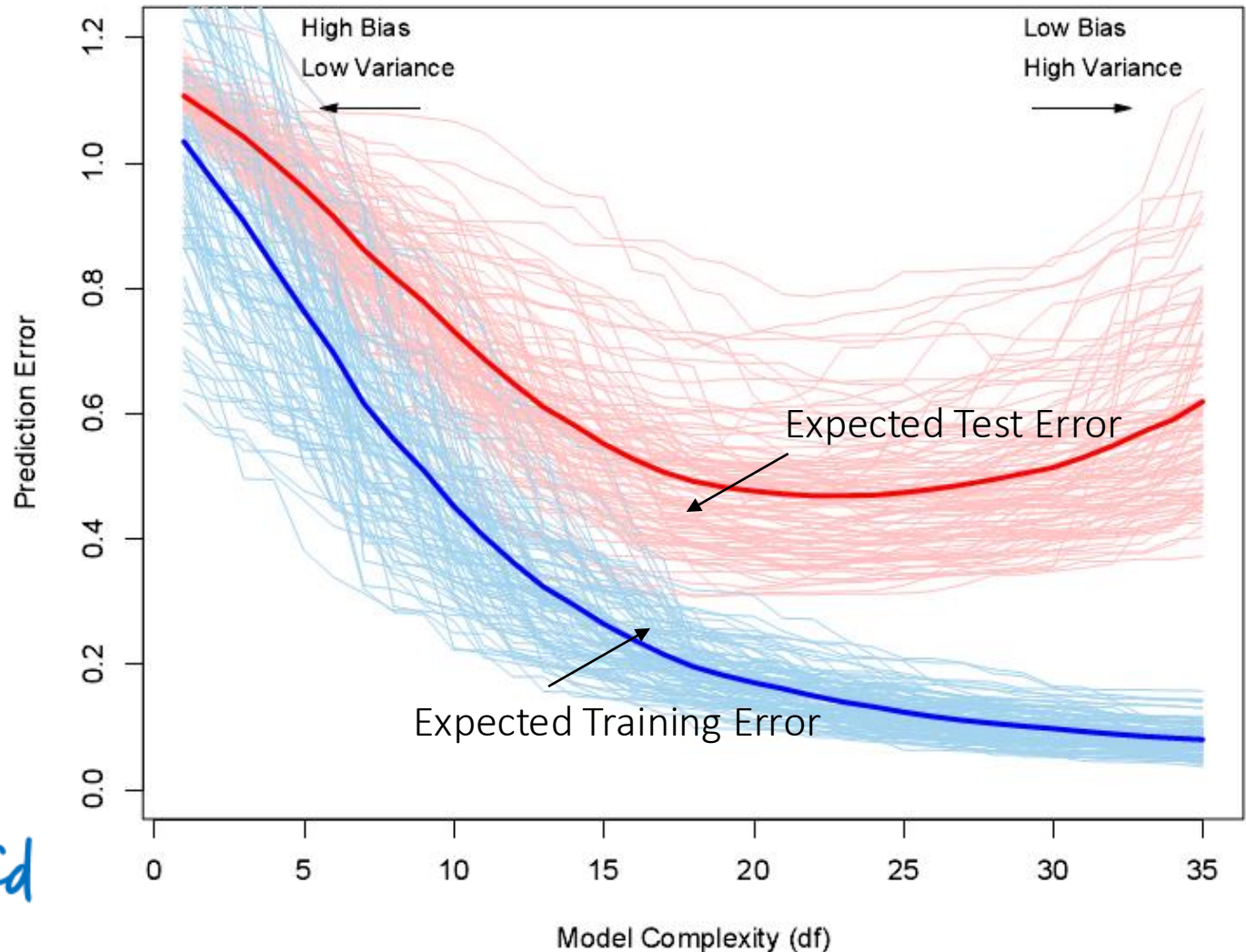
Cross Validation Error as good approximation for Expected Test error → good appx of generalization

See proof
proving
Cross Valid
error as good
approximates
for Expected
Test Error

in Extra

Cross Valid

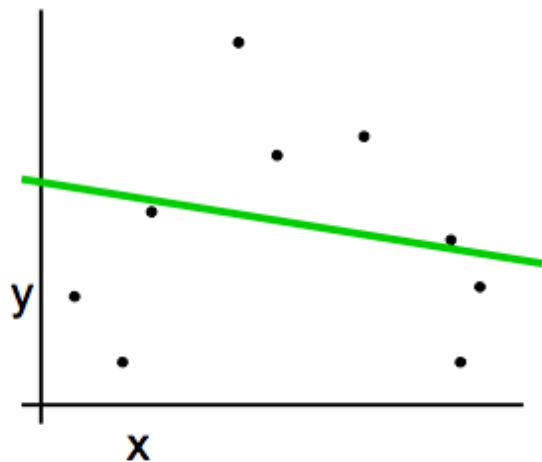
≈ EPE



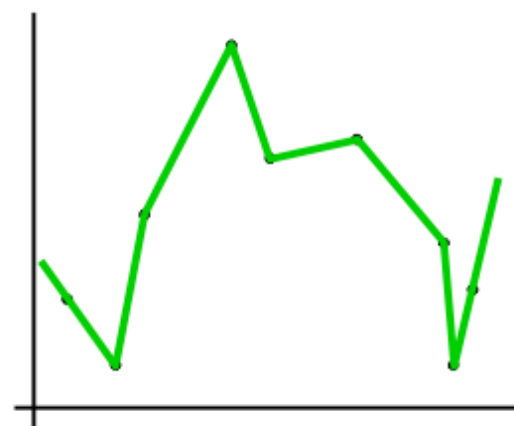
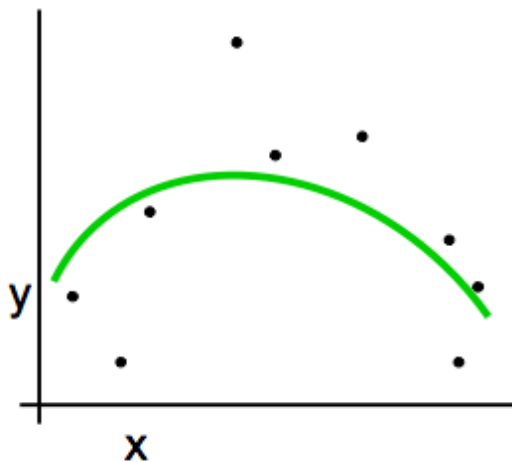
Bias-Variance Trade-off

- Models with too few capacity (parameters) are inaccurate because of a large bias (not enough flexibility).
- Models with too many capacity (parameters) are inaccurate because of a large variance (too much sensitivity to the sample randomness).

Regression: Complexity versus Goodness of Fit



Low Variance /
High Bias



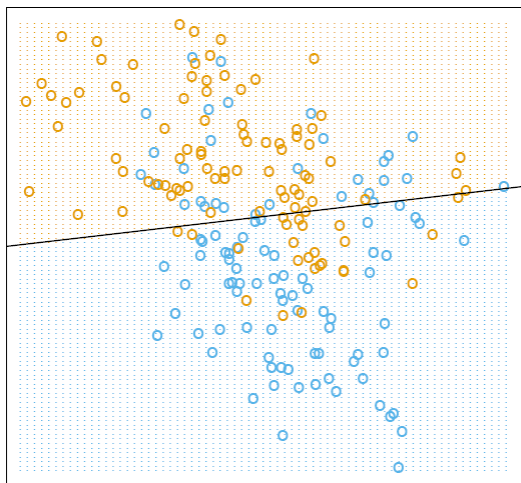
Low Bias
/ High Variance

Highest Bias
Lowest variance
Model complexity = low

Medium Bias
Medium Variance
Model complexity = medium

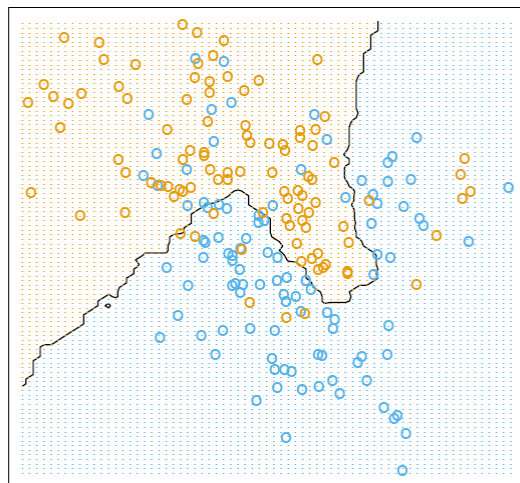
Smallest Bias
Highest variance
Model complexity = high

Classification, Decision boundaries in global vs. local models



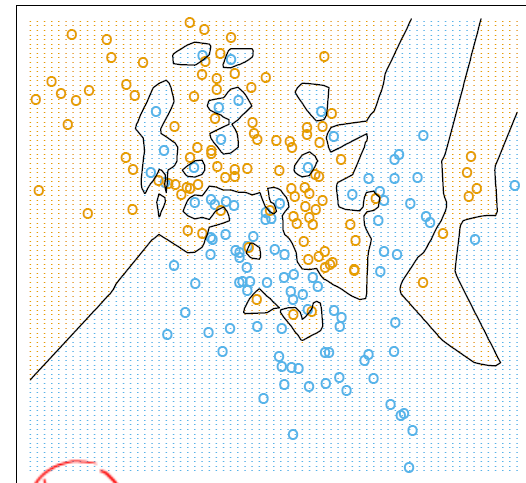
↑
Low Variance /
High Bias

Highest Bias
Lowest variance
Model complexity = low



15-nearest neighbor

Medium Bias
Medium Variance
Model complexity = medium



$k=1$

1-nearest neighbor

↑
Low Bias
/ High Variance

Smallest Bias
Highest variance
Model complexity = high

Thank You



UVA CS 4774: Machine Learning

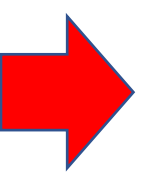
Lecture 9: Bias-Variance Tradeoff

Module III

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Roadmap

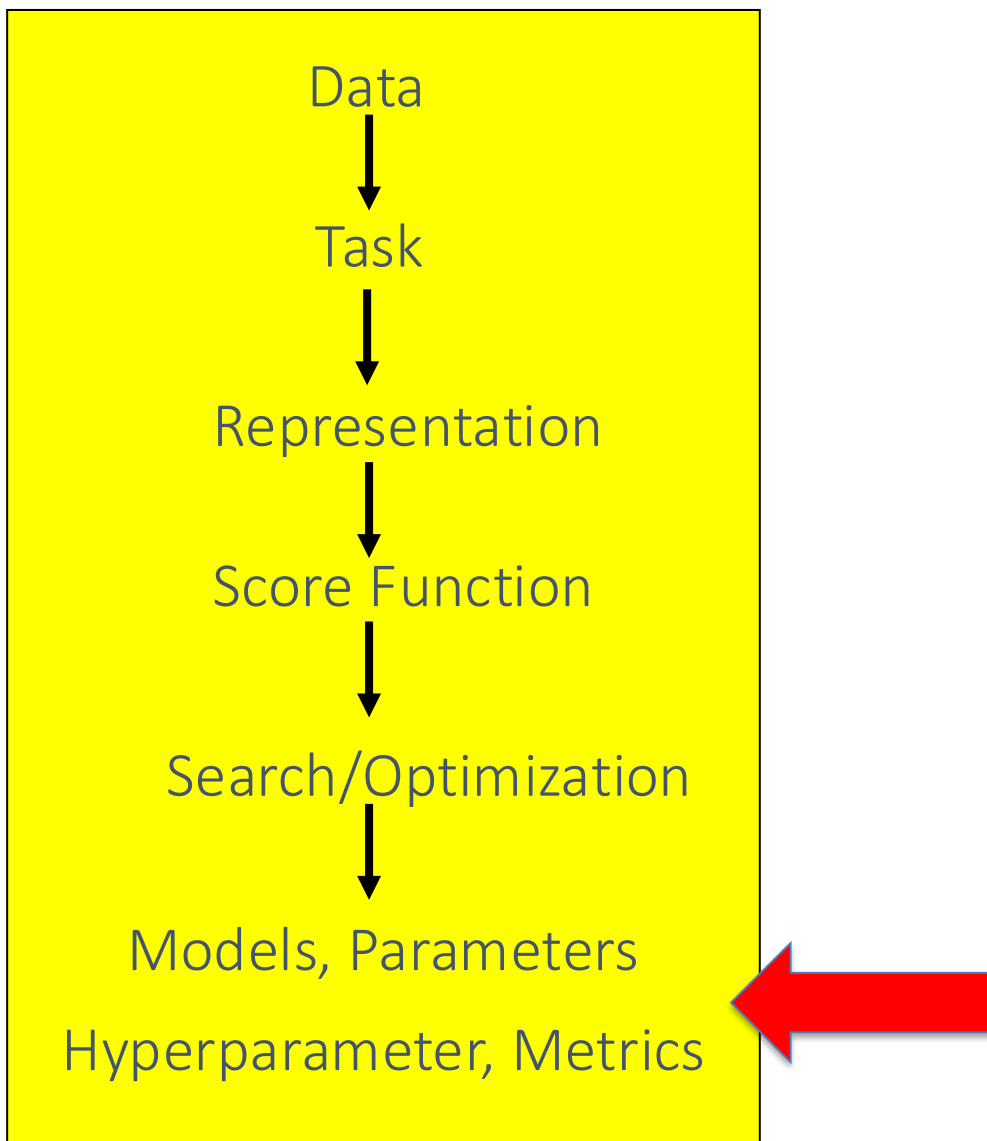
- Bias-variance decomposition
- Bias-Variance Tradeoff / Model Selection
-  • Remedy when Overfit / Underfit

Machine Learning in a Nutshell

ML grew
out of
work in AI

Optimize a
performance
criterion
using
example data
or past
experience,

Aiming to
generalize to
unseen data



Review:

Expected Test Error as $\text{Bias}^2 + \text{variance} + \text{Bayes Error}$

$\Pr(X, Y)$

$$\text{EPE}(x) = \text{noise}^2 + \text{bias}^2 + \text{variance}$$

Unavoidable
error

Error due to
incorrect
assumptions

Error due to variance
of training samples

$(\theta, \bar{\theta})$

$(\bar{\theta}, \hat{\theta}_D)$

Why causes bad generalization?

- Components
 - **Bias**: how much the average model over all training sets differ from the true model?
 - Error due to **inaccurate assumptions/simplifications** made by the model
 - **Variance**: how much models estimated from different training sets **differ from each other**

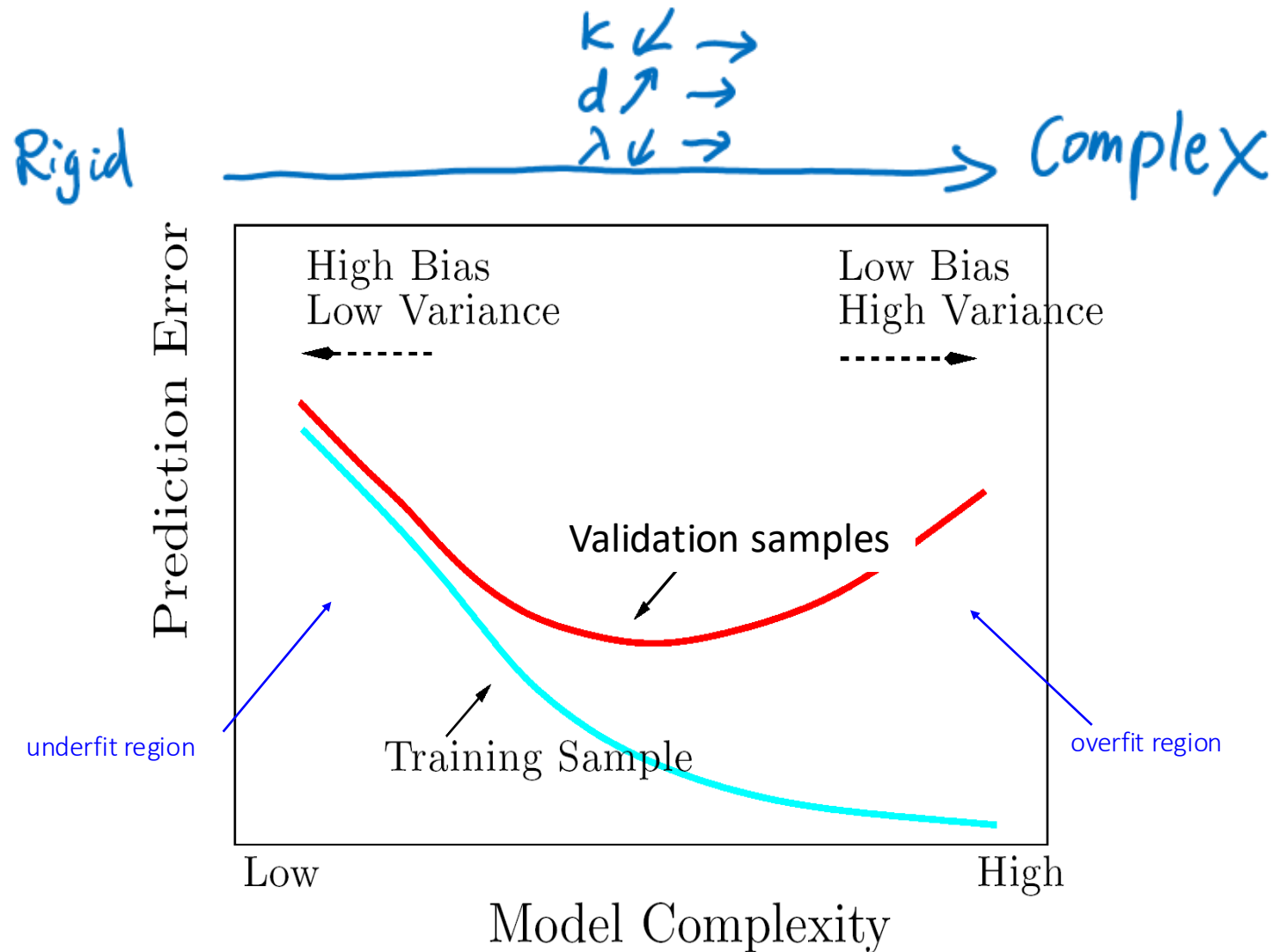
Two Types of bad generalization

- Underfitting: model is too “simple” to represent all the relevant characteristics
 - High bias and low variance
 - High training error and high test error
- Overfitting: model is too “complex” and fits irrelevant characteristics (noise) in the data
 - Low bias and high variance
 - Low training error and high test error

Bias

Variance

If your model complexity can be ordered as X-axis

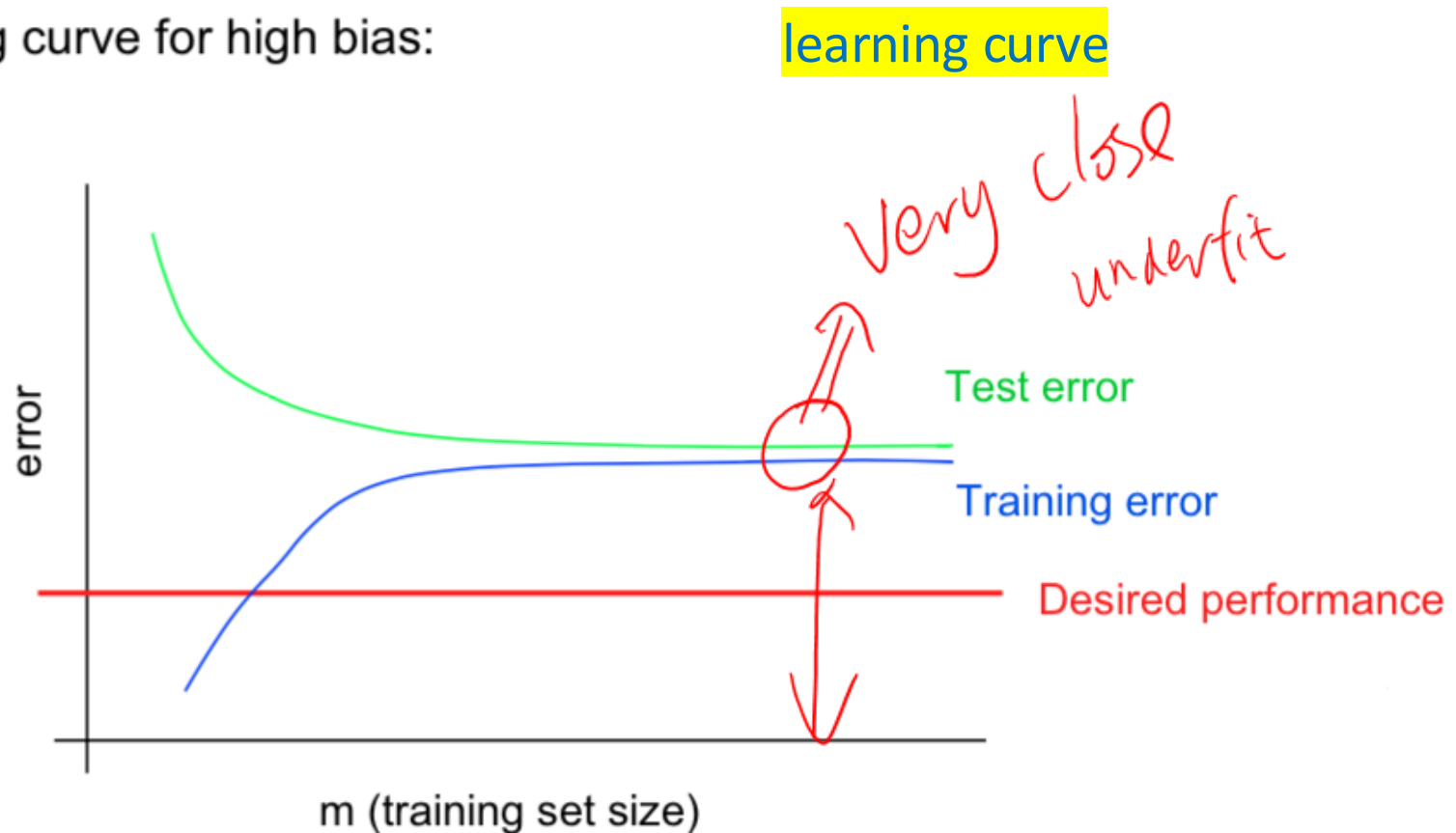


Three types of plots

- (1) Sanity check (S)GD type Optimization
 - Train / Vali Loss vs. Epochs to help you
 - https://scikit-learn.org/stable/auto_examples/linear_model/plot_sgd_early_stopping.html#sphx-glr-auto-examples-linear-model-plot-sgd-early-stopping-py
- (2) Sanity check hyperparameter tuning
 - Train / Vali Loss vs. hyperparameter Values
 - `from sklearn.model_selection import validation_curve`
- (3) Sanity check if your current model overfits or underfits
 - Train / Vali Loss vs. Varying Size of Training
 - https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html#sphx-glr-auto-examples-model-selection-plot-learning-curve-py

(1) If you use learning curve find: Underfitting / High bias / Model too Simple

Typical learning curve for high bias:



- Even training error is unacceptably high.
- Small gap between training and test error.

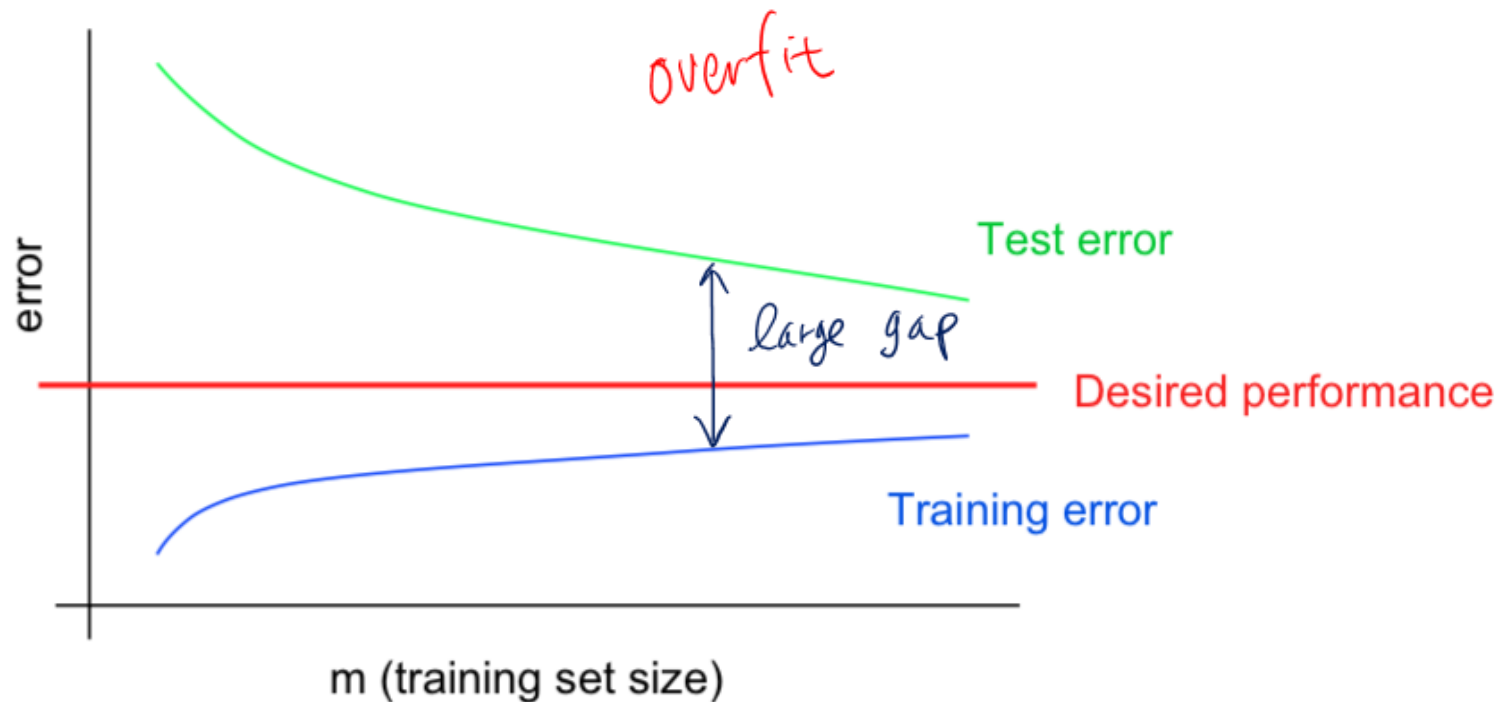
High training error and high test error

How to reduce Model High Bias ?

- E.g.
 - Get additional features
 - Try more complex learner

(2) If you use learning curve find: **Overfitting** / High variance / Model too Complex

Typical learning curve for high variance:



- Test error still decreasing as m increases. Suggests larger training set will help.
- Large gap between training and test error.
- Low training error and high test error

How to reduce Model High Variance?

- Choose a simpler classifier
- Regularize the parameters
- Get more training data
- Try smaller set of features *feature selection* $n < p$
- Try feature engineering
- Try multiple models and then use all as ensemble

(3) How about both high variance / high bias?

- Middle RED:
 - TRUE function
- Error due to bias:
 - How far off in general from the middle red

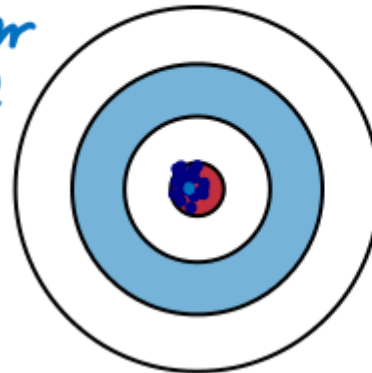
$$E[(\bar{\theta} - \theta)^2]$$

- Error due to variance:
 - How wildly the blue points spread

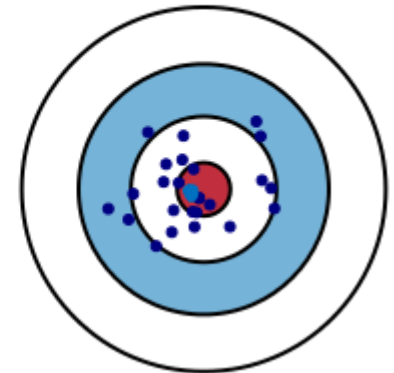
$$E[(\hat{\theta} - \bar{\theta})^2]$$

red
vs.
center
blue
Low Bias

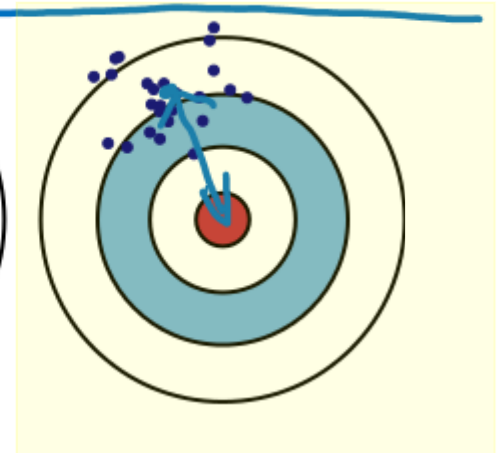
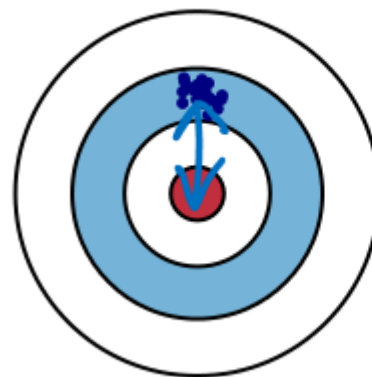
Low Variance



High Variance



High Bias



Is the bias-variance trade off dependent on the number of samples? (EXTRA)

$n \uparrow$
variance $\downarrow$

In the usual application of linear regression, your coefficient estimators are unbiased so sample size is irrelevant. But more generally, you can have bias that is a function of sample size as in the case of the variance estimator obtained from applying the population variance formula to a sample (sum of squares divided by n)....

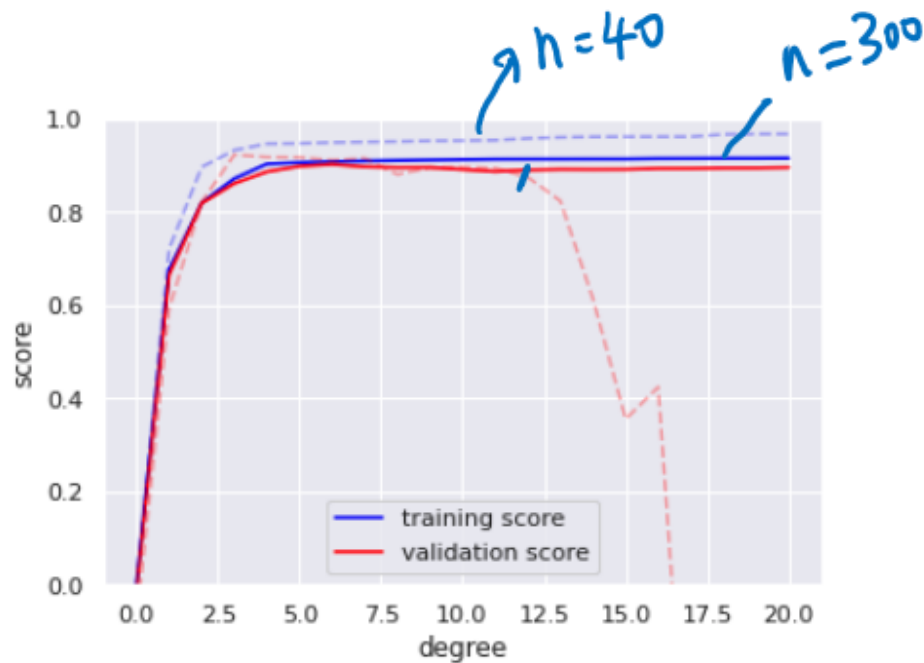
... the bias and variance for an estimator are generally a decreasing function of training size n . Dealing with this is a core topic in nonparametric statistics. For nonparametric methods with tuning parameters a very standard practice is to theoretically derive rates of convergence (as sample size goes to infinity) of the bias and variance as a function of the tuning parameter, and then you find the optimal (in terms of MSE) rate of convergence of the tuning parameter by balancing the rates of the bias and variance. Then you get asymptotic results of your estimator with the tuning parameter converging at that particular rate. Ideally you also provide a data-based method of choosing the tuning parameter (since simply setting the tuning parameter to some fixed function of sample size could have poor finite sample performance), and then show that the tuning parameter chosen this way attains the optimal rate.

Take Away : Three types of plots

- (1) Sanity check (S)GD type Optimization
 - Train / Vali Loss vs. Epochs to help you
 - https://scikit-learn.org/stable/auto_examples/linear_model/plot_sgd_early_stopping.html#sphx-glr-auto-examples-linear-model-plot-sgd-early-stopping-py
- (2) Sanity check hyperparameter tuning
 - Train / Vali Loss vs. hyperparameter Values
 - **from sklearn.model_selection import validation_curve**
- (3) Sanity check if your current model overfits or underfits
 - Train / Vali Loss vs. Varying Size of Training
 - https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html#sphx-glr-auto-examples-model-selection-plot-learning-curve-py

I will Code run

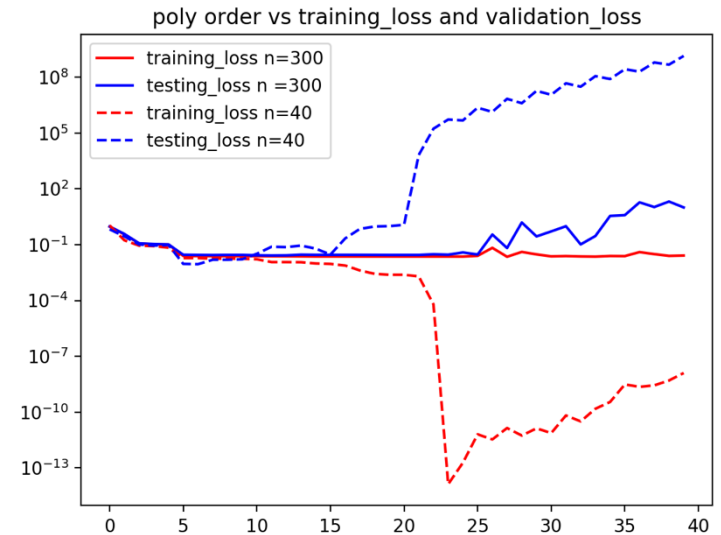
<https://colab.research.google.com/drive/1TvHQoJpYwc5XKz0QGOY3yu1OTljSY5gn?usp=sharing>



(1) Validation curve

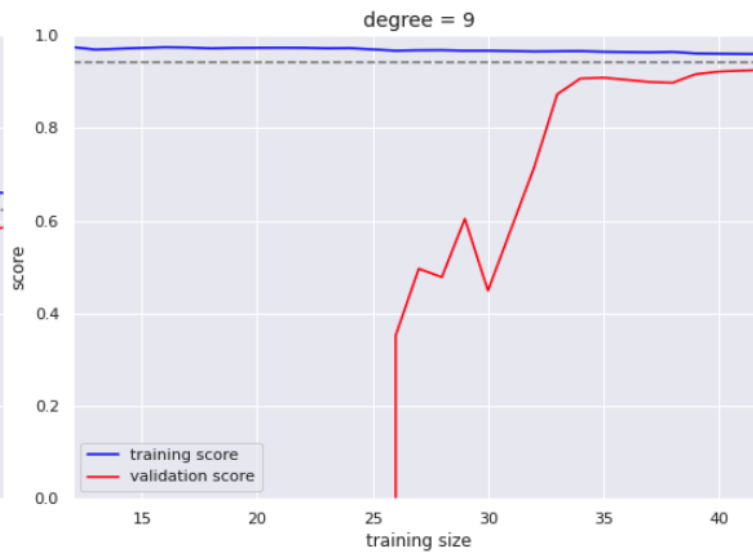
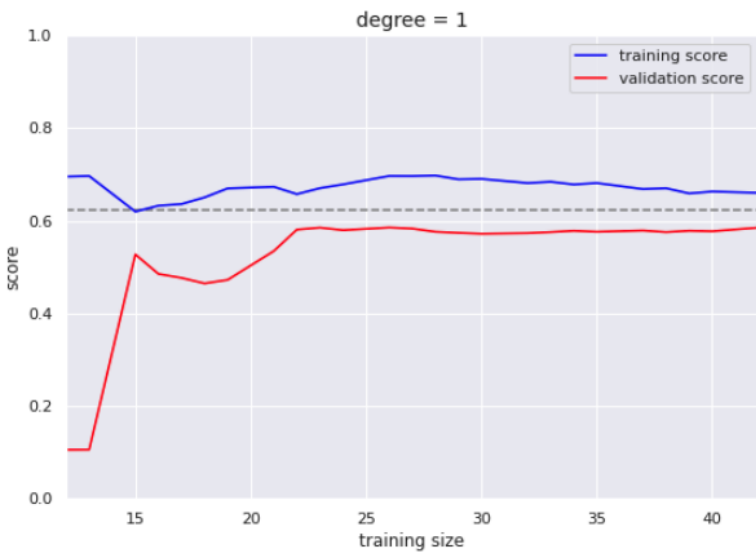
By sklearn Validation_curve function
(normalize all metrics to positive range)

https://scikit-learn.org/stable/modules/model_evaluation.html#the-scoring-parameter-defining-model-evaluation-rules

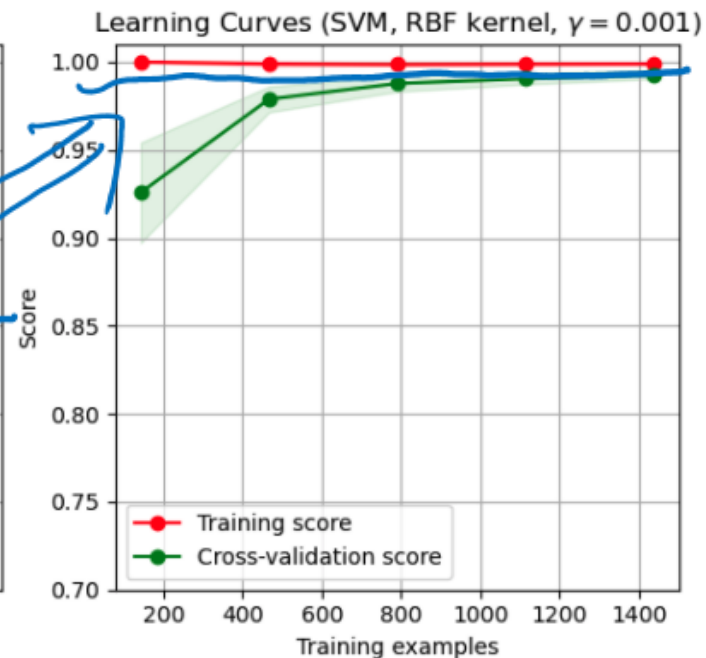
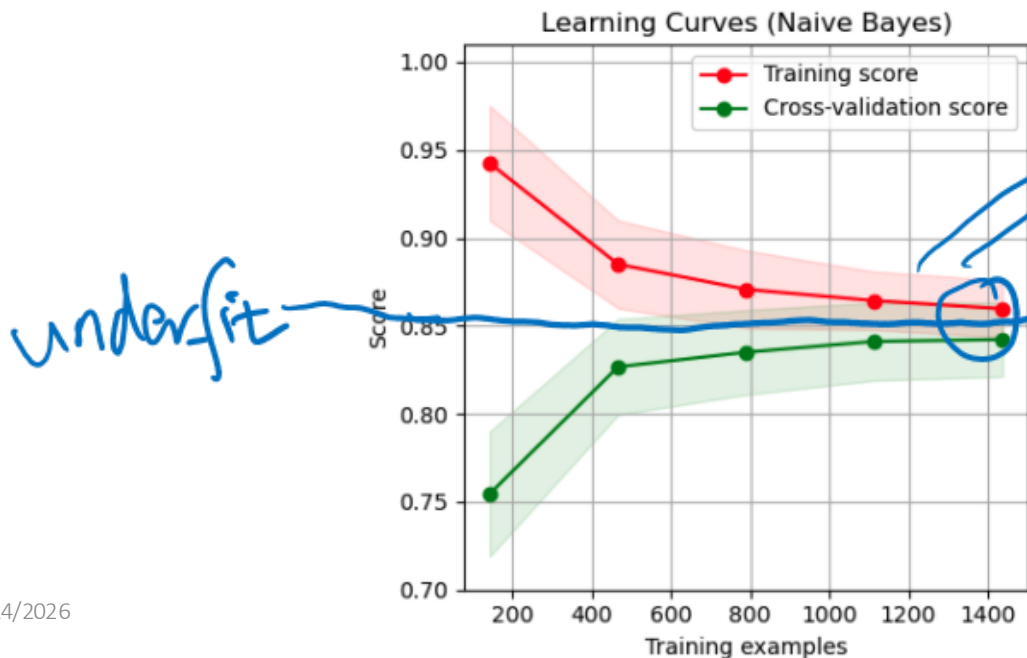


(1) Validation curve

By our HW2 (more close to
modern deep learning
library style)

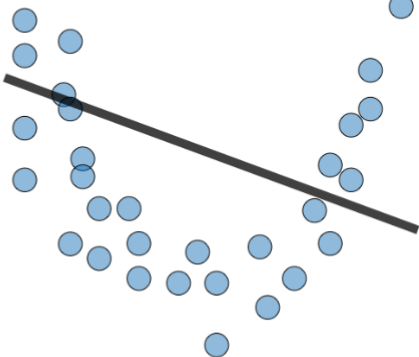
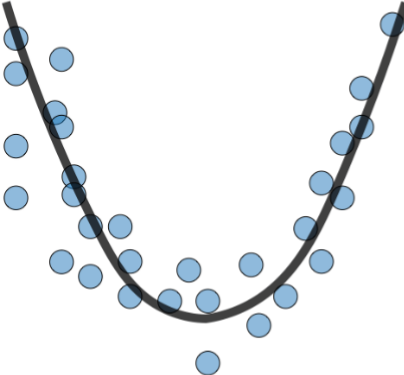
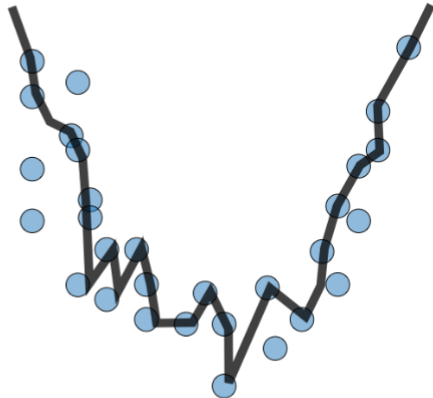
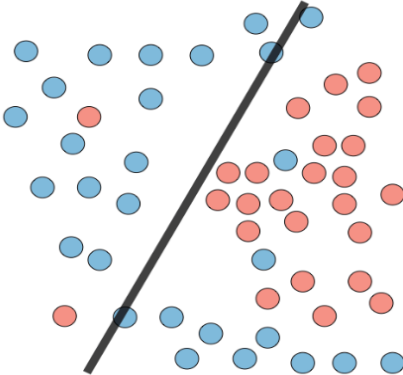
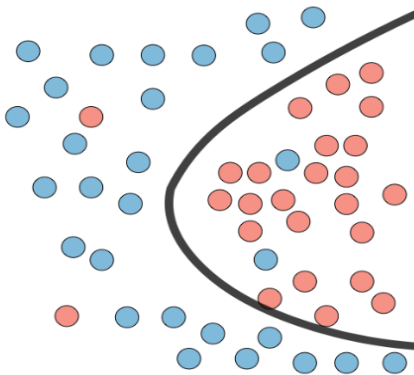
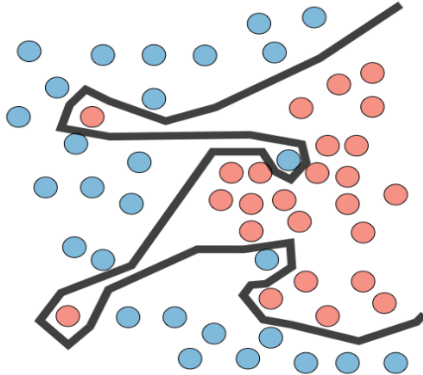


(1) Learning Curves for polynomial regression (up) and classification (down)
/ by scikitlearn



References

- Prof. Tan, Steinbach, Kumar's "Introduction to Data Mining" slide
- Prof. Andrew Moore's slides
- Prof. Eric Xing's slides
- Hastie, Trevor, et al. *The elements of statistical learning*. Vol. 2. No. 1. New York: Springer, 2009.

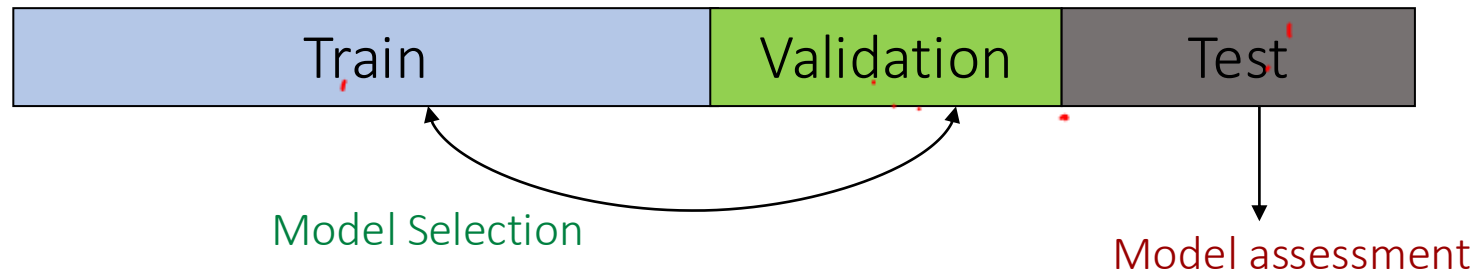
	Underfitting	Just right	Overfitting
Symptoms	- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
Regression			
Classification			
Remedies	- Complexify model - Add more features - Train longer		- Regularize - Get more data - Feature selection

Review: Model Selection and Assessment

- Model Selection
 - Estimating performances of different models to choose the best one
- Model Assessment
 - Having chosen a model, estimating the prediction error on new data

Model Selection and Assessment

- When Data Rich Scenario: Split the dataset



- When Insufficient data to split into 3 parts
 - Approximate validation step analytically
 - AIC, BIC, MDL, SRM
 - Efficient reuse of samples
 - Cross validation, bootstrap

Model Selection (Hyperparameter Tuning) & Model Assessment Pipelines in HW2

- (1) train / Validation / test
- (2) k-CV on train to choose hyperparameter / then test

The battle against overfitting (Extra) :

- Cross validation
- Regularization
- Feature selection
- Model selection --- Occam's razor
- Model averaging
 - The Bayesian-frequentist debate
 - Bayesian learning (weight models by their posterior probabilities)

For instance, if trying to solve “spam detection” using (Extra)

L2 - logistic regression, implemented with gradient descent.

Fixes to try:

If performance is not as desired

- Try getting more training examples.
- Try a smaller set of features.
- Try a larger set of features.
- Try email header features.
- Run gradient descent for more iterations.
- Try Newton’s method.
- Use a different value for λ .
- Try using an SVM.

Fixes high variance.

Fixes high variance.

Fixes high bias.

Fixes high bias.

Fixes optimization algorithm.

Fixes optimization algorithm.

Fixes optimization objective.

Fixes optimization objective.

kNN estimator and L2-EPE

Expected prediction error (EPE)

Consider joint distribution

$$\text{EPE}(f) = \mathbb{E}(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

- For L2 loss: $\text{e.g.} = \int (y - f(x))^2 \Pr(dx, dy)$

under L2 loss, best estimator for EPE (Theoretically) is :

Conditional mean $\hat{f}(x) = \mathbb{E}(Y | X = x)$

e.g. KNN

NN methods are the direct implementation (approximation)

kNN for minimizing EPE

- We know under L2 loss, best estimator for minimize EPE (theoretically) is :

Conditional
mean $f(x) = E(Y | X = x)$

- **Nearest neighbours** assumes that $f(x)$ is well approximated by a locally constant function.

Minimize EPE using L2

- Expected prediction error (EPE) for L2 Loss:

$$\text{EPE}(f) = \mathbb{E}(Y - f(X))^2 = \int (y - f(x))^2 \Pr(dx, dy)$$

- Since $\Pr(X, Y) = \Pr(Y | X) \Pr(X)$, EPE can also be written as

$$\text{EPE}(f) = \mathbb{E}_X \mathbb{E}_{Y|X}([Y - f(X)]^2 | X)$$

- Thus it suffices to minimize EPE pointwise

Best estimator under L2 loss:
conditional expectation

$$f(x) = \arg \min_c \mathbb{E}_{Y|X}([Y - c]^2 | X = x)$$

Conditional
mean

Solution for Regression:

Solution for kNN:



$$f(x) = \mathbb{E}(Y | X = x)$$

Minimize EPE using L2 (another proof)

- Let t be the **true** (target) output and $y(x)$ be our estimate. The **expected squared loss** is

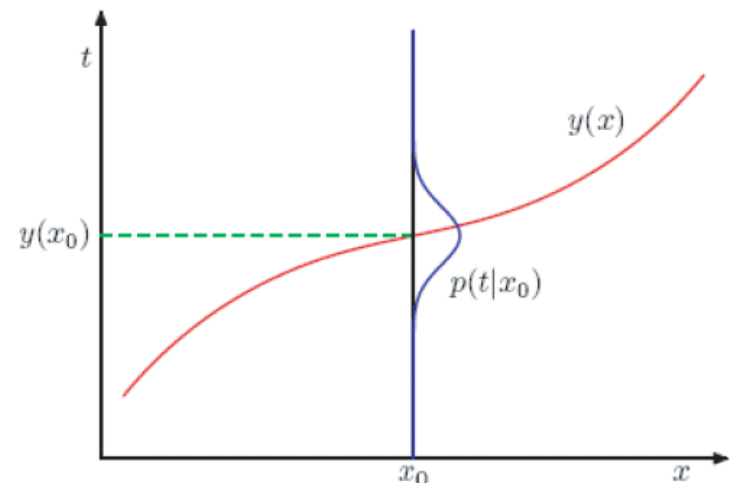
$$\begin{aligned} E(L) &= \iint L(t, y(x)) p(x, t) dx dt \\ &= \iint (t - y(x))^2 p(x, t) dx dt \end{aligned}$$

- Our goal is to choose $y(x)$ that minimize $E(L)$:
 - Calculus of variations:

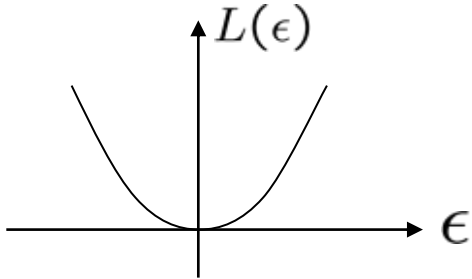
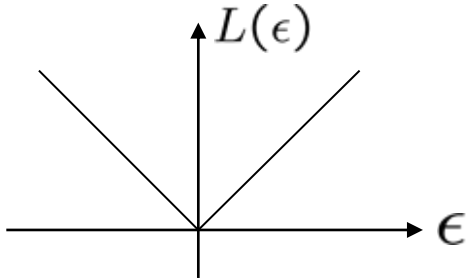
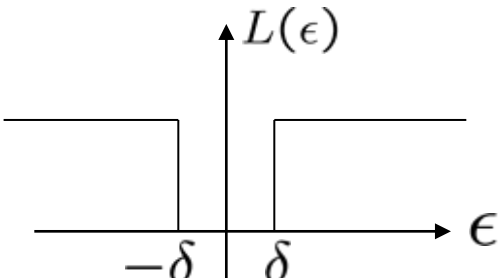
$$\frac{\partial E(L)}{\partial y(x)} = 2 \int (t - y(x)) p(x, t) dt = 0$$

$$\int y(x) p(x, t) dt = \int t p(x, t) dt$$

$$y^*(x) = \int \frac{t p(x, t)}{p(x)} dt = \int t p(t | x) dt = E_{t|x}[t] = E[t | x]$$



Review : EPE with different loss

Loss Function	Estimator $\hat{f}(x)$
L_2 	$\hat{f}(x) = E[Y X = x]$
L_1 	$\hat{f}(x) = \text{median}(Y X = x)$
0-1 	$\hat{f}(x) = \arg \max_Y P(Y X = x)$ (Bayes classifier / MAP)

Expected prediction error (EPE)

Consider joint
distribution

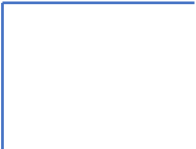
$$\text{EPE}(f) = \mathbb{E}(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

For 0-1 loss: $L(k, \ell) = 1 - d_{kl}$

Bayes Classifier

$$\hat{f}(X) = C_k \text{ if } \Pr(C_k | X = x) \geq \Pr(C_{k'} | X = x) \text{ for all } k' \neq k$$

$$\Pr(C_k | X = x) = \max_{g \in \mathcal{C}} \Pr(g | X = x)$$



More for Overfitting

Bayesian and Frequentist (Extra)

- Frequentist interpretation of probability
 - Probabilities are objective properties of the real world, and refer to limiting relative frequencies (e.g., number of times I have observed heads). Hence one cannot write $P(\text{Katrina could have been prevented}/D)$, since the event will never repeat.
 - Parameters of models are *fixed, unknown constants*. Hence one cannot write $P(\vartheta/D)$ since ϑ does not have a probability distribution. Instead one can only write $P(D/\vartheta)$.
 - One computes point estimates of parameters using various *estimators*, $\vartheta^* = f(D)$, which are designed to have various desirable qualities when *averaged over future data D* (assumed to be drawn from the “true” distribution).
- Bayesian interpretation of probability
 - Probability describes degrees of belief, not limiting frequencies.
 - Parameters of models are *hidden variables*, so one can compute $P(\vartheta/D)$ or $P(f(\vartheta)/D)$ for some function f .
 - One estimates parameters by computing $P(\vartheta/D)$ using Bayes rule:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}$$

Cross Validation and Variance Estimation

- Cross-validation (CV) is quite a general tool for estimating the expected test error (1), that makes minimal assumptions—i.e., it doesn't assume that $Y = f(X) + \varepsilon$ with ε independent of X , it doesn't assume that the training inputs $x_1, \dots, x_n$ are fixed, all it really assumes is that the training samples $(x_1, y_1), \dots, (x_n, y_n)$ are i.i.d.

We split up our training set into K divisions or folds, for some number K ; usually this is done randomly. Write these as $F_1, \dots, F_K$, so $F_1 \cup \dots \cup F_K = \{1, \dots, n\}$. Now for each $k = 1, \dots, K$, we fit our prediction function on all points but those in the k th fold, denoted $\hat{f}^{-(k)}$, and evaluate squared errors on the points in the k th fold,

$$\text{CV}_k(\hat{f}^{-(k)}) = \frac{1}{n_k} \sum_{i \in F_k} (y_i - \hat{f}^{-(k)}(x_i))^2.$$

<http://www.stat.cmu.edu/~ryantibs/statml/review/modelbasics.pdf>

- What is the difference between choosing say $K = 5$ (a common choice) versus $K = n$?
 - When $K = 5$, the function $\hat{f}^{-(k)}$ in each fold k is fit on about $4/5 \cdot n$ samples, and so we are looking at the errors incurred by a procedure that is trained on less data than the full $\hat{f}$ in (1). Therefore the mean of the CV estimate (7) could be off. When $K = n$, this is not really an issue, since each $\hat{f}^{-(k)}$ is trained on $n - 1$ samples
 - When $K = n$, the CV estimate (7) is an average of n extremely correlated quantities; this is because each $\hat{f}^{-(k)}$ and $\hat{f}^{-(\ell)}$ are fit on $n - 2$ common training points. Hence the CV estimate will likely have very high variance. When $K = 5$, the CV estimate will have lower variance, since it is the average of quantities that are less correlated, as the fits $\hat{f}^{-(k)}$, $k = 1, \dots, 5$ do not share as much overlapping training data

This is tradeoff (the bias-variance tradeoff, in fact!). Usually, a choice like $K = 5$ or $K = 10$ is more common in practice than $K = n$, but this is probably an issue of debate

- For K -fold CV, it's can be helpful to assign a notion of variability to the CV error estimate. We argue that

$$\text{Var}(\text{CV}(\hat{f})) = \text{Var}\left(\frac{1}{K} \sum_{k=1}^K \text{CV}_k(\hat{f}^{-(k)})\right) \approx \frac{1}{K} \text{Var}(\text{CV}_1(\hat{f}^{-(1)})). \quad (8)$$

Why is this an approximation? This would hold exactly if $\text{CV}_1(\hat{f}^{-(1)}), \dots, \text{CV}_K(\hat{f}^{-(K)})$ were i.i.d., but they're not. This approximation is valid for small K (e.g., $K = 5$ or 10) but not really for big K (e.g., $K = n$), because then the quantities $\text{CV}_1(\hat{f}^{-(1)}), \dots, \text{CV}_K(\hat{f}^{-(K)})$ are highly correlated

Extra: Practical issues for Cross Validation

- How to decide the values for K in K-CV:
 - Also a bias-variance tradeoff issue
 - Commonly used $K = 10$
 - when data sets are small relative to the number of models that are being evaluated, we need to increase K
 - K needs to be large for the variance to be small enough, but this makes it time-consuming.

Practical issues for CV

- How to decide the values for K in KCV and $a = 1/K$
 - Commonly used $K = 10$ and $a = 0.1$.
 - when data sets are small relative to the number of models that are being evaluated, we need to decrease a and increase K
 - K needs to be large for the variance to be small enough, but this makes it time-consuming.
- Bias-variance trade-off
 - Small a usually lead to low bias. In principle, *LOOCV* provides an almost unbiased estimate of the generalization ability of a classifier, especially when the number of the available training samples is severely limited; but it can also have high variance.
 - Large a can reduce variance, but will lead to under-use of data, and causing high-bias.
- One important point is that the test data D_{test} is never used in CV, because doing so would result in overly (indeed dishonest) optimistic accuracy rates during the testing phase.